

П4. Python и Jupyter для специалистов

Тип: Профессиональный, 24–40 ч · **Связь:** A3, A4 · **Компонент FOLUR:** 4 (развитие потенциала) **Аудитория:** технические специалисты хозяйств и МСБ, агрономы, фермеры с базовой компьютерной грамотностью; уровень ИИ-слоя — **Low-code**.

Модуль превращает специалиста-пользователя в специалиста-автора: вместо ожидания готового отчёта из чужого сервиса вы пишете **свой первый рабочий Python-скрипт под собственный датасет**. Никакой установки ПО: всё выполняется в бесплатном Google Colab. Логика платформы «от инструмента к применению» реализована на реальных данных: тем же датасетом из 80 000 эхолотных промеров, на котором студенты академического модуля A4 строят карту глубин, здесь вы учитесь владеть на прикладном уровне — прочитать, почистить, посчитать, построить график, автоматизировать.

Если программирование вам пока не нужно вовсе — начните с ПЗ «ИИ-инструменты без программирования». П4 — следующая ступень: для тех, кому готовых сервисов уже мало.

Цели модуля

После прохождения модуля слушатель сможет:

1. **[Bloom: понимать]** объяснять, что такое Jupyter-ноутбук, ядро и ячейки, чем воспроизводимый скрипт отличается от ручной работы в Excel и когда переход на Python оправдан для задач хозяйства.
2. **[Bloom: применять]** запускать Python-код в Google Colab без установки ПО: переменные, списки, циклы, функции, чтение файлов, разбор сообщений об ошибках.
3. **[Bloom: применять]** читать, фильтровать, чистить и агрегировать табличные данные в pandas и строить графики в matplotlib — на реальном датасете промеров водохранилища степной зоны Казахстана.
4. **[Bloom: применять]** открывать растровые данные (сетка глубин, спутниковый GeoTIFF) в rasterio/NumPy и запускать пакетную обработку серии файлов одним скриптом.
5. **[Bloom: анализировать]** проверять результат кода — своего и написанного ИИ-ассистентом — по контрольным точкам: число записей, физический диапазон значений, доля удалённых данных, сходимость с ручным расчётом.

Уроки

#	Урок	Трудозатраты ⌚	Ключевой навык
1	Запускаем Python без установки: первый ноутбук в Google Colab	~18 мин чтения + ~25 мин практики	ноутбук, ячейки, базовый синтаксис, чтение ошибок
2	Читаем и чистим 80 000 промеров: pandas и matplotlib	~19 мин чтения + ~30 мин практики	DataFrame, фильтрация, чистка, графики
3	Открываем растры и автоматизируем рутину: rasterio и пакетная обработка	~18 мин чтения + ~30 мин практики	растр как массив, NDVI, цикл по файлам
П	Практикум: первый рабочий скрипт — паспорт водоёма из 80 000 промеров	~75 мин	прикладной продукт модуля
ИИ	ИИ-ассистент в работе: первый скрипт с ИИ-напарником	~45 мин	постановка задачи LLM → исполнение → верификация
А	Итоговая аттестация	~45 мин	банк MCQ + сценарный кейс, порог 75 %

Нагрузка соответствует нормативу платформы: ~2–2,5 часа контента в неделю; каждый микро-урок закрыт квизом сразу после материала.

Прикладной продукт модуля

Первый рабочий Python-скрипт под собственный датасет. В практикуме вы собираете Colab-ноутбук «паспорт водоёма»: загрузка CSV с 80 000 промеров → протоколируемая чистка → статистики и доля мелководий → гистограмма и карта точек → сохранённые очищенный CSV и PNG. Затем — рамка «Проект на ВАШЕЙ

земле»: тот же скрипт адаптируется под любой ваш CSV (журнал урожайности, взвешивания, показания датчиков, собственные промеры). Ноутбук воспроизводим: команда «Restart & Run all» повторяет результат с нуля.

Данные модуля

- **Батиметрические промеры** — уникальный учебный ресурс платформы: `datasets/bathymetry-training-80k.csv` (уровень А: колонки `point_id`, `x_m`, `y_m`, `depth_m`, 80 000 точек, глубины 0–16,1 м, локальная система координат без геопривязки; водоём именуется «водохранилище степной зоны Казахстана»). Полный набор: Zenodo, <DOI будет присвоен при публикации на Zenodo> .
- **Сетка глубин 100 м** (уровень В): `bathymetry-grid-100m.tif` — растровый пример для урока 3.
- **Внешние открытые данные**: Sentinel-2 (Copernicus Data Space / Browser), NASA POWER (агроклиматические ряды), OpenStreetMap.

Правовой режим батиметрических данных — трёхуровневая политика доступа: см. [Датасеты платформы](#). Все задания модуля решаются на открытых уровнях А и В.

ИИ-ассистент в работе · уровень Low-code

«Первый скрипт с ИИ-напарником»: слушатель формулирует задачу по-русски, LLM-ассистент (встроенный помощник Colab или чат-ассистент — Claude и аналоги) пишет код, слушатель исполняет его и **проверяет по контрольным точкам** (число строк, диапазон глубин, сходимость с ручным расчётом). Подробности — в [ИИ-задании](#); рамка ИИ-слоя платформы — [здесь](#).

Правило: ИИ ускоряет — человек верифицирует. Задание выполнимо и без ИИ.

Место в архитектуре платформы

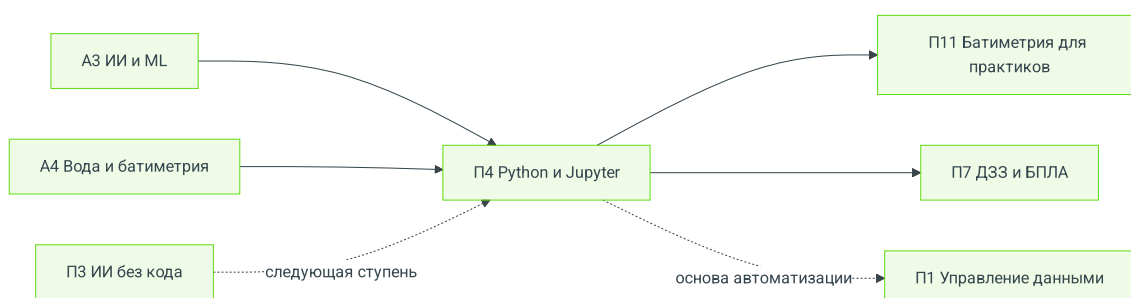


Рис. 1. Связи модуля П4. Alt: П4 опирается на академические модули А3 и А4, продолжает П3 для тех, кому не хватает no-code-сервисов, и готовит слушателя к

обработке данных в П11 и П7.

Пререквизиты: уверенная работа с компьютером и таблицами (Excel/Google Sheets); опыт программирования **не требуется**. Полезно (не обязательно): П1 — организация данных хозяйства, П3 — готовые ИИ-сервисы.

Оценивание

Тренировочные квизы (3–5 MCQ) — после каждой лекции. Итог: индивидуальный вариант из **банка модуля** (порог **75 %**) + сценарный кейс с экспертной проверкой + обязательные артефакты практикума (ноутбук, очищенный CSV, PNG).

Библиография и рамки

Проверенные международные рамки и первичные источники (DOI не указываются там, где не верифицированы):

1. **FAO**. The 10 Elements of Agroecology: Guiding the transition to sustainable food and agricultural systems. FAO, Rome — рамка агроэкологического подхода; данные и их обработка — основа элемента «co-creation and sharing of knowledge».
2. **WOCAT** (World Overview of Conservation Approaches and Technologies) — глобальная база практик устойчивого землепользования; документирование практик требует навыков работы с данными, которые даёт модуль.
3. **ЦУР 15.3 / LDN** (Land Degradation Neutrality, UNCCD) — целевая рамка мониторинга деградации земель; расчёт индикаторов опирается на открытые данные и открытый программный стек.
4. **Каталог кейсов FOLUR Impact Program** (Food Systems, Land Use and Restoration, GEF/UNDP) — источник региональных примеров интегрированного управления ландшафтами.
5. Официальная документация **Python** (docs.python.org), **Project Jupyter** (jupyter.org), **Google Colab** (colab.research.google.com).
6. Официальная документация **pandas** (pandas.pydata.org), **matplotlib** (matplotlib.org), **NumPy** (numpy.org), **rasterio** (rasterio.readthedocs.io).
7. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference* — первичная публикация pandas.
8. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3) — первичная публикация matplotlib.
9. Документация **Copernicus Data Space Ecosystem** (Sentinel-2) и **NASA POWER** (агроклиматические данные) — официальные описания используемых открытых данных.

Правовой режим батиметрических данных: [политика датасетов платформы](#);
канонический каркас модуля: [шаблон](#).

Запускаем Python без установки: первый ноутбук в Google Colab

🕒 **Трудозатраты:** ~18 мин чтения + ~25 мин практики

Модуль **П4: Python и Jupyter для специалистов**, лекция 1 из 3. Профессиональный уровень. Проект UNDP-KAZ, FOLUR.

Цели лекции и место в модуле

Эта лекция снимает главный барьер: «программирование — не для меня». За один урок вы запустите настоящий Python-код в браузере — без установки чего-либо на компьютер, — освоите минимальный синтаксис, которого хватает для реальных задач хозяйства, и научитесь читать сообщения об ошибках вместо того, чтобы их бояться. Всё остальное в модуле — pandas, графики, растры, пакетная обработка — строится на этом фундаменте.

После изучения лекции слушатель сможет:

1. **[Bloom: понимать]** объяснять, когда переход от Excel и готовых сервисов к Python оправдан, а когда — нет; что такое ноутбук, ячейка и ядро.
2. **[Bloom: применять]** создавать и исполнять ячейки в Google Colab, сохранять ноутбук, загружать в сессию файл с данными.
3. **[Bloom: применять]** использовать переменные, списки, словари, условия, циклы и функции для простых расчётов по данным хозяйства.
4. **[Bloom: анализировать]** разбирать traceback (сообщение об ошибке): находить строку-виновника и различать типовые ошибки новичка.

Границы лекции. Внутри: рабочее место (Colab/Jupyter), базовый синтаксис Python, чтение файлов стандартными средствами, культура работы с ошибками. Снаружи: таблицы pandas и графики — в [лекции 2](#); растры и пакетная обработка — в [лекции 3](#); написание кода с помощью ИИ-ассистента — в [ИИ-задании](#); машинное обучение и теория методов — в академическом модуле [А3](#). Если вам нужен только готовый мониторинг полей без кода — это модуль [П3](#).

1. Зачем специалисту код, когда есть Excel и готовые сервисы

1.1 Три причины, по которым таблицы перестают справляться

Excel и Google Sheets — отличные инструменты, и модуль П1 строит учёт хозяйства именно на них. Но у ручной работы с таблицами есть три ограничения, с которыми рано или поздно сталкивается каждый технический специалист.

Масштаб. Таблица с журналом взвешиваний на тысячу строк листается глазами. Файл эхолотной съёмки на 80 000 строк — уже нет: Excel его откроет, но каждая ручная операция (найти подозрительные значения, пересчитать колонку) превращается в многоминутное ожидание и риск случайно испортить данные. Датасеты дистанционного зондирования в таблицы не помещаются вовсе.

Воспроизводимость. Отчёт, собранный ручными операциями «скопировал — вставил — отсортировал — удалил строки», невозможно повторить точно. Через полгода вы не вспомните, какие строки удалили и почему. Скрипт — это протокол: тот же код на тех же данных всегда даёт тот же результат, и коллега может проверить каждый шаг.

Автоматизация. Если операция повторяется — каждую неделю приходит новая выгрузка, каждый сезон новый журнал, — ручная обработка съедает часы. Скрипт, написанный один раз, обрабатывает и один файл, и сто файлов одинаково: меняется только имя файла. К пакетной обработке мы придём в лекции 3.

Есть и честная обратная сторона: первые недели Python медленнее Excel. Кривая обучения реальна, и для разовой задачи на 50 строк заводить ноутбук не нужно. Практическое правило: **если задачу нужно решить один раз — решайте как умеете; если каждую неделю — пишите скрипт.**

1.2 Почему именно Python и где П4 в системе модулей

Python — самый распространённый язык анализа данных: для него написаны все ключевые открытые библиотеки — pandas (таблицы), matplotlib (графики), rasterio и GeoPandas (геоданные), scikit-learn (машинное обучение). Он бесплатен, работает на любой системе и не привязан ни к одному поставщику — это принцип всего открытого стека платформы (Python, QGIS, Jupyter). Именно на Python работают академические модули А3 (машинное обучение) и А4 (батиметрия и мониторинг воды): П4 даёт вам прикладное ядро того же инструмента, чтобы результаты «старших» модулей были вам доступны на практике.

Подробнее о выборе между готовыми сервисами и собственным анализом — в модуле П3: там критически сравниваются платные и бесплатные платформы мониторинга.

2. Jupyter и Colab: рабочее место без установки

2.1 Что такое ноутбук

Jupyter-ноутбук — это документ, в котором чередуются два типа ячеек: **текстовые** (заголовки, пояснения, выводы — обычный текст с разметкой Markdown) и **кодовые** (Python-код, который можно исполнить прямо в документе). Под кодовой ячейкой появляется её вывод: числа, таблицы, графики. Получается «живой отчёт»: методика, код и результат в одном файле с расширением `.ipynb`.

Исполняет код **ядро** (kernel) — фоновый Python-процесс, привязанный к ноутбуку. Важнейшее свойство ядра: оно **помнит состояние**. Если в первой ячейке вы создали переменную, во второй она доступна. Отсюда и главная ловушка новичка: ячейки можно исполнять в любом порядке, и ноутбук, исполненный «вразнобой», может давать результат, который не повторится при честном запуске сверху вниз. Правило профессионала: перед тем как считать результат готовым — **Restart & Run all** (перезапустить ядро и исполнить всё сверху вниз). Прошло — ноутбук воспроизводим; упало — вы нашли скрытую зависимость от порядка исполнения.

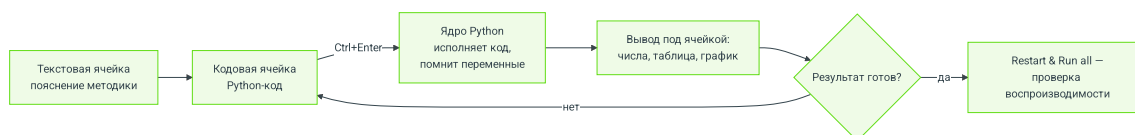


Рис. 1. Цикл работы в ноутбуке: код исполняется ядром, вывод появляется под ячейкой, финальная проверка — полный прогон сверху вниз. Alt: блок-схема из шести узлов — текстовая ячейка, кодовая ячейка, ядро, вывод, проверка готовности и обязательный полный перезапуск перед сдачей результата.

2.2 Google Colab: ноутбук в браузере

Google Colab (colab.research.google.com) — бесплатный сервис, который запускает Jupyter-ноутбуки на серверах Google. Для специалиста это означает: **не нужно ничего устанавливать** — нужен браузер и аккаунт Google. Основные библиотеки (pandas, matplotlib, NumPy) уже предустановлены; недостающие ставятся одной командой `!pip install rasterio` прямо в ячейке.

Практический минимум навигации: «Файл → Создать блокнот» — новый ноутбук; «+ Код» / «+ Текст» — новые ячейки; `Ctrl+Enter` — исполнить ячейку; значок папки слева — файлы сессии, куда можно перетащить свой CSV. Ноутбуки сохраняются на ваш Google Диск и открываются с любого компьютера.

Ограничения тоже нужно знать. Сессия Colab **временная**: после закрытия или долгого простоя виртуальная машина обнуляется, и загруженные в сессию файлы исчезают

(сам ноутбук — нет, он на Диске). Поэтому данные либо загружают в начале каждой сессии, либо подключают Google Диск командой из двух строк. Для работы нужен устойчивый интернет; при слабой связи в селе разумнее короткие сессии: загрузили данные, прогнали скрипт, скачали результат.

Текстовые ячейки — не украшение, а рабочая часть ноутбука. В них пишут постановку задачи, решения по данным («нулевые значения сохранены, потому что...»), выводы по графикам. Разметка Markdown проста: # — заголовок, ****жирный****, — — пункт списка. Хороший ориентир: человек, открывший ваш ноутбук, должен понять задачу и выводы, читая **только текстовые ячейки**, — код при этом служит проверяемым приложением. Именно такие ноутбуки принимает аттестация модуля и именно так оформлены ноутбуки-образцы платформы.

2.3 Локальная альтернатива: JupyterLab

Когда интернет ненадёжен или данные нельзя загружать в облако, тот же ноутбук запускается локально: дистрибутив Python + JupyterLab устанавливаются бесплатно на обычный офисный компьютер (проект Jupyter, jupyter.org). Интерфейс и формат `.ipynb` идентичны — ноутбук из Colab откроется в JupyterLab и наоборот. В этом модуле все задания рассчитаны на Colab как на нулевой порог входа; локальная установка — по желанию, инструкция есть в официальной документации Jupyter.

Подробнее о том, где хранить данные хозяйства и как организовать резервное копирование, — в модуле [П1](#).

3. Базовый синтаксис за один вечер

Python создавался читаемым, и это видно: большинство конструкций понятны без объяснений. Разберём минимум, который покрывает реальные задачи специалиста. Каждый фрагмент ниже стоит исполнить в своём Colab-ноутбуке — чтение кода без исполнения не учит.

3.1 Переменные, числа, строки

```
glubina = 3.4          # число с дробной частью (float)
tochek = 80000         # целое число (int)
vodoem = "водохранилище" # строка (str)

ploshchad_m2 = 10000
obem = glubina * ploshchad_m2
print(f"Объём призмы: {obem} м³") # f-строка подставляет значения
```

Переменная — именованная ячейка памяти; знак `=` означает «положить значение». Имена пишите осмысленно и латиницей: `glubina_m` лучше, чем `x`, — через месяц вы скажете себе спасибо. F-строка (`f"..."`) — стандартный способ собрать текст с числами: внутри фигурных скобок можно даже форматировать — `{obem:.1f}` выведет одну цифру после запятой.

3.2 Списки и словари

```
glubiny = [0.27, 2.87, 3.4, 16.1, 0.0] # список — упорядоченный набор
print(len(glubiny)) # 5 — длина
print(max(glubiny)) # 16.1
print(glubiny[0]) # 0.27 — нумерация с нуля!

zapis = {"point_id": 1, "x_m": 34202.2, "y_m": 37336.1, "depth_m": 0.27}
print(zapis["depth_m"]) # доступ по имени поля
```

Список (`list`) хранит значения по порядку, словарь (`dict`) — пары «ключ → значение», как строка таблицы с именованными колонками. Обратите внимание: **нумерация в Python начинается с нуля** — первый элемент списка имеет индекс 0. Это источник половины ошибок новичков и одновременно соглашение, единое для всего языка.

3.3 Условия и циклы

```
melkovodie = 0
for d in glubiny: # цикл: перебираем значения по одному
    if d < 2.0: # условие
        melkovodie = melkovodie + 1

dolya = melkovodie / len(glubiny)
print(f"Доля точек мельче 2 м: {dolya:.0%}")
```

Цикл `for` повторяет блок для каждого элемента; условие `if` исполняет блок только когда выражение истинно. Обратите внимание на **отступы**: в Python вложенность блоков обозначается сдвигом на четыре пробела, и неровный отступ — синтаксическая ошибка, а не оформление. Colab расставляет отступы автоматически, но при копировании кода из мессенджеров они часто ломаются — это первое, что стоит проверить.

3.4 Функции и импорт

```
def dolya_melkovodiy(spisok, porog=2.0):
    """Доля значений мельче порога (по умолчанию 2 м)."""
    melkih = 0
    for d in spisok:
        if d < porog:
```

```

        melkih += 1
    return melkih / len(spisok)

print(dolya_melkovodiy(glubiny))      # порог 2 м
print(dolya_melkovodiy(glubiny, 5.0)) # порог 5 м

import statistics                      # модуль стандартной библиотеки
print(statistics.median(glubiny))

```

Функция упаковывает расчёт в переиспользуемый блок с именем: написали один раз — вызываете сколько угодно, меняя аргументы. Именно так растут ваши личные инструменты: сегодня функция «доля мелководий», завтра — «паспорт водоёма».

`import` подключает готовые модули: `statistics`, `csv`, `math` входят в стандартную библиотеку, а мощные внешние библиотеки (`pandas`, `matplotlib`) подключаются так же — с ними мы работаем со следующей лекции.

Разберите строку `def dolya_melkovodiy(spisok, porog=2.0):` по частям: `def` объявляет функцию, в скобках — аргументы, а `porog=2.0` — аргумент **со значением по умолчанию**: его можно не указывать при вызове. Строка в тройных кавычках сразу под заголовком — докстринг, короткое описание назначения; команда `help(dolya_melkovodiy)` покажет его любому пользователю вашей функции, включая вас через полгода. `return` отдаёт результат наружу — без него функция посчитает и промолчит: типичная ошибка новичка, когда «функция вроде работает, а результата нет».

4. Файлы, ошибки и культура работы

4.1 Читаем настоящий CSV

CSV (comma-separated values) — текстовый формат таблиц: первая строка — имена колонок, дальше — данные через запятую. Именно в CSV распространяется учебный датасет платформы. Прочитать его можно даже без `pandas` — стандартным модулем `csv`:

```

import csv

n = 0
maks = 0.0
with open("bathymetry-training-80k.csv") as f:
    for row in csv.DictReader(f):
        d = float(row["depth_m"])
        n += 1
        if d > maks:
            maks = d

print(f"Записей: {n}, максимальная глубина: {maks} м")

```

Здесь видно всё изученное сразу: файл открывается конструкцией `with open(...)` (она гарантирует закрытие файла), цикл перебирает строки-словари, `float()` превращает текст `"16.1"` в число. На 80 000 строк этот код отработывает за секунды — то, что в Excel потребовало бы осторожной ручной работы. В [лекции 2](#) pandas сожмёт этот фрагмент до одной строки, но понимать, что происходит «под капотом», вы теперь будете всегда.

4.2 Traceback: сообщение об ошибке — это подсказка, а не приговор

Код падает у всех, включая профессионалов; разница в реакции. Python при ошибке печатает **traceback** — трассировку: цепочку вызовов и в последней строке — тип и описание ошибки. Читать traceback нужно **снизу вверх**: последняя строка говорит «что случилось», строки выше — «где».

Типовые ошибки новичка и их расшифровка:

Сообщение	Что означает	Типичная причина
<code>NameError: name 'glubina' is not defined</code>	переменная не существует	опечатка в имени; ячейка с определением не исполнена
<code>FileNotFoundError</code>	файл не найден	файл не загружен в сессию Colab; опечатка в имени; сессия обнулилась
<code>IndentationError</code>	неверный отступ	код скопирован с поломанными пробелами
<code>TypeError: '<' not supported between 'str' and 'float'</code>	сравнение текста с числом	забыли <code>float()</code> при чтении из CSV
<code>IndexError: list index out of range</code>	индекс за пределами списка	забыли, что нумерация с нуля

Алгоритм действий: прочитать последнюю строку → найти в traceback номер строки своего кода → проверить очевидное (опечатки, порядок исполнения ячеек, загружен ли файл). Если за пять минут причина не нашлась — скопируйте traceback целиком в поиск или ИИ-ассистенту: разбор чужих ошибок отложен десятилетиями, ваша почти

наверняка типовая. Как правильно работать с ИИ-подсказками и проверять их — в [ИИ-задании модуля](#).

4.3 Три правила культуры, которые дороже синтаксиса

Сырые данные неприкосновенны. Скрипт читает исходный файл и пишет результат в **новый** файл. Никогда не перезаписывайте исходник: чистка, фильтрация, пересчёт — всё порождает новые версии. Это правило платформа повторяет в каждом модуле работы с данными — от [П1](#) до [A4](#).

Ноутбук должен проходить Restart & Run all. Результат, который не воспроизводится полным прогоном сверху вниз, — не результат (§2.1).

Комментируйте намерение, а не механику. Комментарий `# порог 2 м — граница мелководья для зарастания` полезен; комментарий `# прибавляем единицу` — шум. Ваш главный читатель — вы сами через полгода.

Региональный кейс

Ситуация: Агроном хозяйства в Акмолинской области хочет сравнить влагообеспеченность двух последних вегетационных сезонов: сложить суточные осадки за май–август каждого года. Метеопоста в хозяйстве нет, но открытый сервис NASA POWER (power.larc.nasa.gov) бесплатно выдаёт суточные агроклиматические ряды (осадки, температура) для любой точки по координатам — достаточно указать точку своего района в веб-форме и скачать CSV. Excel для одной пары сезонов справился бы, но агроном хочет решение, которое следующей весной отработает за минуту.

Данные: NASA POWER Data Access Viewer — суточные ряды параметра «осадки» (PRECTOTCORR) для точки в своём районе, экспорт в CSV; учебный датасет платформы для тренировки чтения CSV.

Задача:

1. Скачать CSV с суточными осадками за два сезона и загрузить его в сессию Colab.
2. Циклом по строкам просуммировать осадки за май–август каждого года (по колонкам года и месяца в файле NASA POWER).
3. Оформить расчёт функцией `summa_osadkov(fajl, god)` и вывести результат f-строкой для обоих лет.

Разбор: Ход решения повторяет §4.1: `csv.DictReader`, преобразование `float()`, условие на месяц `if 5 <= mesyac <= 8`, накопление суммы в переменной. Конкретные миллиметры зависят от выбранной точки и лет — их вычисляете вы, а не лекция;

проверка правдоподобия — по известной климатологии региона: Северный Казахстан — зона недостаточного увлажнения, сезонные суммы измеряются десятками–сотнями миллиметров, и результат в тысячи миллиметров означает ошибку в коде (например, суммирование всех месяцев или дублирование строк). Такая проверка «числа на здравый смысл» — первый навык верификации, который в **ИИ-задании** станет обязательным шагом.

Закрепление и применение



Попробуйте в Colab

Ноутбук лекции (<URL будет назначен при создании репозитория ноутбуков>) содержит все фрагменты §3–§4. Минимальное задание на 15 минут: создайте новый ноутбук, повторите §3.1–3.3 руками (не копированием!), затем сломайте код тремя способами — опечатка в имени переменной, неверный отступ, индекс за пределами списка — и прочитайте все три traceback по алгоритму §4.2.



Границы применимости

Colab — облачный сервис: данные загружаются на серверы Google, сессии временные, нужен интернет. Конфиденциальные данные хозяйства (финансы, персональные данные работников) в облачный ноутбук загружать нельзя без решения владельца данных; учебный датасет платформы для этого и обезличен. Python не заменяет Excel в задачах «посмотреть и поправить пару ячеек» — он выигрывает на повторяемых и объёмных задачах (§1.1).



Что это даёт вашему хозяйству

Через неделю практики вы перестанете зависеть от единственного человека «который умеет в компьютер»: суммы по журналам, проверка выгрузок техники, пересчёт норм — задачи на 10 строк кода. Ноутбук с пояснениями станет живой инструкцией: новый сотрудник открывает его и видит и методику, и расчёт, и результат.

? Кейс для размышления

Бухгалтер хозяйства раз в квартал вручную сверяет две выгрузки (весовая и учётная система) — день работы, и дважды находились расхождения. Стоит ли переводить сверку на Python-скрипт? Оцените по правилу §1.1 (повторяемость, объём, цена ошибки) и подумайте, что в этой задаче останется человеку даже при идеальном скрипте.

Словарь урока

Термины, которыми вы будете пользоваться до конца модуля, — одной таблицей для быстрого возврата:

Термин	Значение
Ноутбук	документ <code>.ipynb</code> , в котором чередуются текстовые и кодовые ячейки; методика, код и результат живут в одном файле
Ядро (kernel)	фоновый Python-процесс, исполняющий код ячеек и помнящий переменные между ними
Restart & Run all	полный перезапуск ядра и прогон всех ячеек сверху вниз — обязательная проверка воспроизводимости результата
CSV	текстовый формат таблиц: первая строка — имена колонок, значения разделены запятыми
f-строка	запись <code>f"..."</code> для подстановки значений переменных в текст; <code>{обем:.1f}</code> форматирует вывод
Докстринг	описание назначения функции в тройных кавычках сразу под <code>def</code> ; показывается командой <code>help()</code>
Traceback	сообщение об ошибке; читается снизу вверх: последняя строка — «что случилось», строки выше — «где»

Проверь себя

Самопроверка · вариант слушателя конспект · вопросов: 3 · порог зачёта: 75%

1. Ноутбук даёт правильный результат при выполнении ячеек вразнобой, но падает при «Restart & Run all». Что это означает?

- ☐ Ошибка в сервисе Colab — нужно просто перезапустить сессию
- ☐ Так и должно быть: полный прогон нужен только для очень больших ноутбуков
- ☐ Ноутбук зависит от порядка исполнения ячеек и невоспроизводим — нужно исправить порядок определений
- ☐ Ядро сломалось и требует переустановки Python

2. В списке `glubiny = [0.27, 2.87, 3.4]` выражение `glubiny[1]` вернёт...

- ☐ 0.27 — первый элемент списка
- ☐ Ошибку `IndexError`
- ☐ 3.4 — последний элемент
- ☐ 2.87 — нумерация элементов начинается с нуля

3. Код сравнения глубины с порогом упал с ошибкой `TypeError: '<' not supported between 'str' and 'float'`. Наиболее вероятная причина?

- ☐ Файл CSV повреждён и его нужно скачать заново
- ☐ Порог задан слишком большим числом
- ☐ Значение из CSV прочитано как текст — забыли преобразование `float()`
- ☐ В Colab не установлена библиотека `pandas`

Проверить

Контрольные вопросы

1. Что такое ядро ноутбука и почему оно «помнит» переменные между ячейками? (*Bloom: понимать*)
2. Перечислите три ограничения ручной работы в Excel, которые снимает скрипт. (*Bloom: знать/понимать*)
3. Напишите функцию, которая по списку глубин возвращает долю значений глубже заданного порога, и проверьте её на списке из §3.2. (*Bloom: применять*)

Что дальше?

Вы умеете исполнять код и читать файлы — пора работать с настоящей таблицей. В [лекции 2 «Читаем и чистим 80 000 промеров»](#) библиотека pandas за одну строку прочитает весь учебный датасет, а matplotlib нарисует первую карту точек. Если хотите сразу увидеть, куда всё движется, — загляните в [практикум](#): его первый шаг вы уже способны выполнить.

Читаем и чистим 80 000 промеров: pandas и matplotlib

🕒 **Трудозатраты:** ~19 мин чтения + ~30 мин практики

Модуль **П4: Python и Jupyter для специалистов**, лекция 2 из 3. Профессиональный уровень. Проект UNDP-KAZ, FOLUR.

Цели лекции и место в модуле

Это центральная лекция модуля: здесь появляется главный рабочий инструмент специалиста по данным — библиотека pandas — и первый настоящий датасет: 80 000 сырых эхолотных промеров водохранилища степной зоны Казахстана, собранных полевыми экспедициями исполнителя. Данные реальные, со всеми дефектами реальной съёмки, и путь «прочитать → посмотреть → отфильтровать → почистить → посчитать → нарисовать → сохранить», который вы пройдёте, — это универсальный конвейер для любой вашей будущей таблицы: журнала урожайности, выгрузки датчиков, собственных промеров.

После изучения лекции слушатель сможет:

1. **[Bloom: понимать]** объяснять, что такое DataFrame, чем он отличается от списка словарей и листа Excel и почему операции в pandas применяются к целым колонкам сразу.
2. **[Bloom: применять]** читать CSV в DataFrame, осматривать данные (`head`, `info`, `describe`), выбирать колонки и фильтровать строки булевыми масками, агрегировать через `groupby`.
3. **[Bloom: применять]** проводить протоколируемую чистку: находить пропуски, дубликаты и физически невозможные значения, сохранять очищенную версию отдельным файлом.
4. **[Bloom: анализировать]** строить и интерпретировать гистограмму и диаграмму рассеяния в matplotlib: читать форму распределения глубин и структуру галсов съёмки.

Границы лекции. Внутри: pandas для табличных данных, базовая чистка, гистограмма и диаграмма рассеяния. Снаружи: базовый синтаксис Python и работа в Colab — в [лекции 1](#); растры и пакетная обработка — в [лекции 3](#); полный конвейер с сохранением артефактов — в [практикуме](#). Профессиональная фильтрация

выбросов по локальной статистике соседей и интерполяция в карту глубин — предмет академического модуля [A4](#) и профмодуля [П11](#): здесь мы делаем честную базовую чистку, а не гидрографическую обработку.

1. DataFrame: таблица, у которой есть память и мускулы

1.1 Знакомимся с датасетом

Учебный датасет `datasets/bathymetry-training-80k.csv` — случайный сабсет в 80 000 точек из полного набора полевых промеров (~540 000 записей; полный набор публикуется на Zenodo: <DOI будет присвоен при публикации на Zenodo>). Четыре колонки: `point_id` — номер записи, `x_m` и `y_m` — координаты в **локальной метрической системе**, `depth_m` — глубина в метрах. Геопривязка удалена безвозвратно, название и расположение водоёма не раскрываются — это уровень А трёхуровневой политики данных платформы (см. [Датасеты](#)); для всех учебных задач реальное положение водоёма и не нужно: геометрия и глубины полностью сохранены.

Читается всё это одной строкой:

```
import pandas as pd

df = pd.read_csv("bathymetry-training-80k.csv")
df.head()          # первые 5 строк — быстрый взгляд
```

`df` — это **DataFrame**: таблица с именованными колонками и пронумерованными строками. Внешне похоже на лист Excel, но с принципиальной разницей: вы не редактируете ячейки мышкой, а описываете операции над целыми колонками, и pandas исполняет их над всеми 80 000 строк одновременно. Одна колонка (`df["depth_m"]`) называется Series — с ней работают те же приёмы.

Практическая деталь для ваших собственных файлов: экспорт из русскоязычного Excel часто использует точку с запятой как разделитель и запятую в дробях. Обе проблемы решаются аргументами чтения: `pd.read_csv(fajl, sep=";", decimal=",")`. Если после чтения вся таблица оказалась одной колонкой со странным именем — почти наверняка дело в разделителе, а не в «битом файле».

1.2 Три команды осмотра

Первое, что делает специалист с новой таблицей, — разведочный осмотр тремя командами:

```
df.shape          # (80000, 4) — строк и колонок
df.info()         # имена колонок, типы, число непустых значений
```

```
df.describe() # статистики по числовым колонкам
```

`info()` отвечает на вопросы «всё ли прочиталось числами?» и «есть ли пропуски?». Типы данных pandas стоит знать в лицо: `int64` — целые числа (номера точек), `float64` — числа с дробной частью (координаты, глубины), `object` — текст. Если колонка, которая обязана быть числовой, получила тип `object`, — где-то в данных мусор: текстовый комментарий оператора посреди чисел, запятая вместо точки в дробях, слово «нет» вместо пустой ячейки. Это надо найти и обезвредить **до** любых расчётов, иначе первая же арифметическая операция упадёт или, что хуже, молча посчитает не то.

`describe()` выдаёт минимум, максимум, среднее, медиану и квартили — и это уже проверка данных на здравый смысл. Для нашего датасета: глубины лежат в диапазоне от 0 до 16,1 м, средняя — около 3,4 м, медиана — около 3,0 м. Среднее заметно больше медианы — распределение скошено: мелководных точек больше, чем глубоких, что типично для равнинного водохранилища с обширными заливами. Уже три команды дали содержательный вывод о водоёме — без единого графика.

Выработайте рефлекс: эти три команды выполняются для **каждой** новой таблицы, которая попадает вам в руки, до любых действий с ней. Осмотр занимает минуту, а экономит часы: почти все неприятные сюрпризы данных (не тот разделитель, лишняя строка заголовка, перепутанные колонки, коды ошибок датчика вроде `-999`) видны уже на этом шаге.

Подробнее о том, как этот же датасет превращается в карту глубин и расчёт объёма, — в [лекции 2 модуля А4](#).

1.3 Конвейер лекции целиком

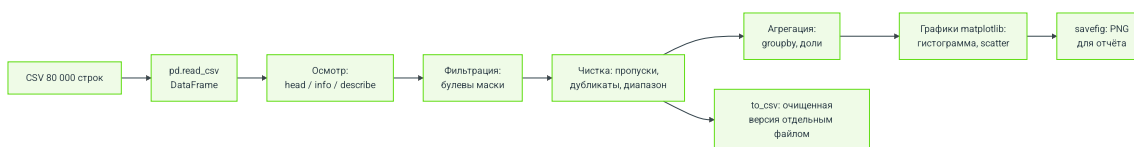


Рис. 1. Конвейер обработки таблицы: от сырого CSV до очищенного файла и графиков. Alt: блок-схема — CSV читается в DataFrame, проходит осмотр, фильтрацию, чистку и агрегацию; результаты сохраняются как новый CSV и PNG-графики.

2. Выбираем и фильтруем: булевы маски

2.1 Колонки и маски



```
df["depth_m"]          # одна колонка (Series)
df[["x_m", "y_m"]]     # несколько колонок — список имён

melko = df[df["depth_m"] < 2.0]      # строки мельче 2 м
nuli  = df[df["depth_m"] == 0.0]     # нулевые отсчёты
```

Выражение `df["depth_m"] < 2.0` вычисляется для всех строк сразу и даёт колонку из `True / False` — **булеву маску**. Подстановка маски в `df[...]` оставляет только строки с `True`. Это главный приём pandas: любые «найти все записи, где...» — одна строка кода. Условия комбинируются: `&` — «и», `|` — «или», каждое условие в скобках:

```
sredniye = df[(df["depth_m"] >= 2.0) & (df["depth_m"] < 5.0)]
dolya_nulej = len(nuli) / len(df)
print(f"Нулевых отсчётов: {len(nuli)} ({dolya_nulej:.1%})")
```

На учебном датасете нулевых отсчётов около 4–5 % — и это не техническая мелочь, а содержательный вопрос: часть нулей — настоящий урез воды у берега, часть — потери дна эхолотом (заросли, аэрация, мелководье). Отличить одно от другого без пространственного анализа соседних точек нельзя — поэтому в этой лекции мы нули **не удаляем**, а честно считаем и документируем. Профессиональные правила разделения — в [A4](#).

2.2 Сортировка и агрегация

```
df.sort_values("depth_m", ascending=False).head(10)  # 10 самых глубоких точек

df["zona"] = pd.cut(df["depth_m"],
                    bins=[-0.01, 2, 5, 10, 17],
                    labels=["0–2 м", "2–5 м", "5–10 м", ">10 м"])
df.groupby("zona", observed=True)["depth_m"].agg(["count", "mean"])
```

`pd.cut` нарезает непрерывную глубину на зоны, `groupby` считает статистику по каждой зоне — за две строки получается таблица «сколько точек и какая средняя глубина в каждой зоне», готовая в отчёт. Тот же приём работает для любых категорий: урожайность по полям, привесы по группам, осадки по месяцам. Обратите внимание: мы **создали новую колонку** `df["zona"]` — DataFrame расширяется на лету, исходный CSV при этом не тронут.

Логика `groupby` заслуживает отдельного абзаца, потому что это самая мощная идея pandas. Схема всегда трёхчастная: **разбить** таблицу на группы по значению колонки → **применить** к каждой группе функцию (счёт, среднее, максимум) → **собрать** результаты в итоговую таблицу. Один вызов заменяет вложенный цикл с ручным накоплением сумм, который вы написали бы средствами лекции 1, — и работает быстрее и надёжнее. Если вы освоите только две конструкции из всей библиотеки —

булевы маски и `groupby`, — вы уже закроете большинство табличных задач хозяйства.

Ещё один будничный инструмент — `value_counts()`: одна команда

`df["zona"].value_counts()` выдаёт частоты категорий и мгновенно отвечает на вопросы вида «сколько записей по каждому полю/бригаде/прибору». Для журнала с текстовой колонкой «культура» это первый шаг осмотра: опечатки («пшеница» и «пшеница») сразу видны как отдельные строки частот — и это тоже находка чистки, а не мелочь.

3. Чистка: протокол вместо «на глазок»

3.1 Пропуски и дубликаты

```
df.isna().sum()      # пропуски по каждой колонке
df.duplicated().sum() # полностью одинаковые строки
df = df.drop_duplicates() # удалить дубликаты (если есть)
```

Пропуск (`NaN`) — пустая ячейка: датчик не сработал, оператор не заполнил. Стратегий три: удалить строки (`dropna()`), заполнить значением (`fillna()`), оставить и учитывать. Универсально правильной нет — решение зависит от задачи и **обязательно фиксируется** в тексте ноутбука. Дубликаты в выгрузках техники и журналах — частый гость (повторная выгрузка, двойное нажатие); проверка на них — обязательный пункт чистки любой таблицы.

3.2 Физический диапазон

Самая надёжная чистка — по физике задачи. Глубина не бывает отрицательной; глубина больше максимальной для данного водоёма — тоже дефект (эхо от рыбы, двойное отражение). Порог сверху берут из априорного знания о водоёме — с запасом, а не по самим данным, иначе можно отрезать законный максимум:

```
was = len(df)
clean = df[(df["depth_m"] >= 0) & (df["depth_m"] <= 17.0)].copy()
print(f"Удалено {was - len(clean)} строк из {was}")
```

Мелочь в коде выше, о которую спотыкаются новички: метод `.copy()` в конце строки создаёт независимую копию отобранных строк. Без него pandas может вернуть «представление» исходной таблицы и выдать предупреждение `SettingWithCopyWarning` при следующей записи в колонку. Правило простое: отфильтровали строки для дальнейшей работы — добавьте `.copy()`.

Две детали профессиональной культуры. Первая: скрипт **сам печатает протокол** — сколько строк удалено и по какому правилу; чистка без протокола не отличается от порчи данных. Вторая: результат сохраняется **новым файлом**, сырые данные неприкосновенны (правило из [лекции 1](#), §4.3):

```
clean.to_csv("bathymetry-clean.csv", index=False)
```

На учебном датасете правило диапазона не удалит ничего — глубины уже лежат в 0–16,1 м, — и это тоже результат: «проверили — дефектов диапазона нет» является строкой протокола, а не потерянным временем. В ваших собственных выгрузках этот же фильтр регулярно ловит `-999` (код ошибки датчика), опечатки операторов и сбои приборов.

3.3 Чего базовая чистка не делает

Честность метода — знать его пределы. Фильтр диапазона не найдёт **правдоподобный мусор**: одиночную точку 12 м среди соседних 4-метровых (эхо от косяка рыбы) — значение возможное, но в этом месте ложное. Такие выбросы ловятся только локальной статистикой соседних точек, и это уже гидрографическая обработка — модуль [A4](#) для теории, [П11](#) для практики на собственных съёмках. Базовая чистка этой лекции — необходимый первый эшелон, после которого данные пригодны для статистик и графиков, но ещё не для официальной карты глубин.

4. matplotlib: два графика, которые видят то, что скрывают числа

4.1 Гистограмма — форма распределения

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(clean["depth_m"], bins=40, edgecolor="white")
ax.set_xlabel("Глубина, м")
ax.set_ylabel("Число промеров")
ax.set_title("Распределение глубин: 80 000 промеров")
fig.savefig("gistogramma-glubin.png", dpi=150)
```

Гистограмма делит диапазон глубин на корзины (`bins`) и показывает, сколько промеров попало в каждую. На учебном датасете вы увидите скошенное вправо распределение (что предсказали числа из §1.2) и пик у нуля — те самые нулевые отсчёты из §2.1: график мгновенно делает видимым то, что в таблице пришлось бы искать. Три обязательных элемента любого графика в отчёт: подписи осей **c**

единицами измерения, заголовок, читаемый размер. `savefig` сохраняет PNG — файл появляется в панели файлов Colab, откуда его можно скачать.

Число корзин — параметр с характером: слишком мало корзин (`bins=5`) замажет пик нулей, слишком много (`bins=400`) превратит форму в гребёнку шума. Правило практики — начать с нескольких десятков и подвигать: форма распределения должна быть устойчива к разумной смене `bins`. Чем это лучше диаграммы в Excel? Тем же, чем скрипт лучше ручной работы: график строится кодом, а значит, следующая выгрузка данных даст обновлённый график **тем же кодом** — без повторного накликивания мастера диаграмм.

4.2 Диаграмма рассеяния — карта точек съёмки

У нас есть координаты `x_m`, `y_m` — значит, можно нарисовать точки в плане, раскрасив по глубине:

```
fig, ax = plt.subplots(figsize=(9, 7))
sc = ax.scatter(clean["x_m"], clean["y_m"],
                c=clean["depth_m"], s=1, cmap="viridis_r")
fig.colorbar(sc, label="Глубина, м")
ax.set_xlabel("x, м (локальная СК)")
ax.set_ylabel("y, м (локальная СК)")
ax.set_aspect("equal")          # метр по x равен метру по y
ax.set_title("Промеры: карта точек, цвет — глубина")
fig.savefig("karta-tochek.png", dpi=150)
```

Этот график — почти карта: видна форма акватории, параллельные линии галсов лодки с эхолотом, тёмные глубокие участки и светлые мелководья. «Почти» — потому что координаты локальные: наложить его на реальную карту нельзя, и для учебных задач не нужно. Обратите внимание на `set_aspect("equal")`: без него `masshtab` осей различен и форма водоёма искажается — частая ошибка в отчётах. Аргумент `s=1` делает точки мелкими: на 80 000 точек крупные маркеры сливаются в пятно.

Диаграмма рассеяния — это и инструмент контроля чистки: если после фильтрации на карте точек появились «дыры» в середине акватории — фильтр удалил не мусор, а данные.

Пара слов о цвете. `cmap` — цветовая шкала; `viridis_r` выбрана не случайно: шкала равномерна по восприятию (одинаковые перепады глубин выглядят одинаково контрастно), различима при цветоаномалии и корректно печатается в оттенках серого; суффикс `_r` разворачивает её так, чтобы глубокое было тёмным — интуитивная конвенция батиметрии. Радужные шкалы, любимые старыми программами, создают ложные визуальные границы там, где в данных их нет, — в отчётах их лучше избегать.

Подробнее об оформлении карт по правилам картографии (легенда, масштаб, компоновка) — в модулях [П2](#) и [А2](#); matplotlib здесь — инструмент быстрой диагностики, а не картографии.

Региональный кейс

Ситуация: Фермерское хозяйство в Костанайской области арендует пруд-накопитель для полива и водопоя и заказало эхолотную съёмку подрядчику. Подрядчик прислал CSV на десятки тысяч строк и короткое письмо: «сырые данные, чистите сами».

Техническому специалисту хозяйства нужно за час дать руководителю ответ: корректны ли данные, какова структура глубин и какую долю акватории занимают мелководья, склонные к зарастанию и быстрому прогреву. В качестве тренировочного полигона специалист использует учебный датасет платформы — по формату и дефектам он идентичен реальной выгрузке эхолота.

Данные: `datasets/bathymetry-training-80k.csv` (уровень A: `point_id`, `x_m`, `y_m`, `depth_m`, 80 000 точек) — см. [Датасеты платформы](#).

Задача:

1. Прочитать CSV, выполнить осмотр тремя командами (§1.2) и зафиксировать: число записей, диапазон и медиану глубин, число пропусков и дубликатов.
2. Посчитать долю нулевых отсчётов и долю точек мельче 2 м; разбить глубины на зоны `pd.cut` и получить таблицу «зона → число точек» (§2.2).
3. Построить гистограмму глубин и карту точек (§4); по карте оценить, где сосредоточены мелководья — по периметру или отдельными заливами.

Разбор: Ожидаемый ход: осмотр подтверждает 80 000 записей без пропусков, глубины 0–16,1 м; нулевых отсчётов — около двадцатой части датасета, точек мельче 2 м — порядка десятой части (точные значения слушатель вычисляет сам — они и есть ответ руководителю). Ключевой аналитический шаг — не числа, а их интерпретация: нулевые отсчёты нельзя молча удалять (часть из них — берег, §2.1), а «долю мелководий по точкам» нельзя выдавать за «долю площади» — промеры расположены неравномерно вдоль галсов, и честный пересчёт в площади требует интерполяции ([А4](#)). Грамотный вывод специалиста звучит так: «по точкам съёмки — такая-то доля, оценка по площади требует построения сетки». Умение видеть эту разницу и есть цель кейса.

Закрепление и применение



Попробуйте в Colab

Ноутбук лекции (<URL будет назначен при создании репозитория ноутбуков>) содержит весь конвейер §1–§4 на учебном датасете. Минимальное задание на 20 минут: выполните осмотр тремя командами, посчитайте долю точек глубже 10 м, постройте обе картинки §4 и добейтесь, чтобы карта точек не была искажена (`set_aspect("equal")`).



Границы применимости

Базовая чистка (диапазон, пропуски, дубликаты) не заменяет гидрографическую обработку: правдоподобные локальные выбросы остаются в данных (§3.3), а доли по точкам — не доли по площади. Учебный датасет — в локальной системе координат: накладывать его на реальные карты нельзя. `pandas` держит таблицу в оперативной памяти: сотни тысяч строк — легко, сотни миллионов — уже инженерная задача за рамками модуля.



Что это даёт вашему хозяйству

Замените в коде лекции имя файла и имена колонок — и тот же конвейер обработает журнал урожайности по полям, выгрузку комбайна или показания датчиков влажности: осмотр, чистка с протоколом, зоны, гистограмма. Это и есть ваш «первый скрипт под собственный датасет» — прикладной продукт модуля, который вы соберёте в [практикуме](#).



Кейс для размышления

В выгрузке датчика влажности почвы 3 % значений — ровно -999, ещё 2 % — пропуски. Коллега предлагает: «заполним всё средним, чтобы график был гладкий». Какие два разных решения требуются для этих двух групп и почему «заполнить средним» опасно для последующего расчёта норм полива? Рассуждайте от §3.1–3.2.

Словарь урока

Термин	Значение
DataFrame	таблица <code>pandas</code> с именованными колонками и пронумерованными строками; операции применяются к колонкам целиком

Термин	Значение
Series	одна колонка DataFrame; поддерживает те же маски и статистики
Булева маска	колонка <code>True / False</code> , полученная сравнением; подстановка в <code>df[...]</code> отбирает строки
NaN	«not a number» — пропуск, пустая ячейка; ищется командой <code>isna()</code>
groupby	схема «разбить → применить → собрать»: статистика по группам одной командой
pd.cut	нарезка непрерывной величины на зоны-категории по заданным границам
value_counts()	частоты значений колонки; первый инструмент осмотра текстовых колонок и поиска опечаток
.copy()	независимая копия отобранных строк; страховка от предупреждений при дозаписи колонок

Проверь себя

Самопроверка · вариант слушателя **конспект** · вопросов: 4 · порог зачёта: 75%

1. Почему нулевые глубины в датасете нельзя просто удалить одной командой?

- ☐ Часть нулей — настоящие точки у уреза воды, а часть — потери дна эхолотом; без анализа соседних точек их не различить
- ☐ Нулевые значения нужны для правильной работы гистограммы
- ☐ Ноль — служебное значение pandas, его удаление ломает DataFrame
- ☐ Удаление нулей запрещено политикой данных платформы

2. Что вернёт выражение `df[df["depth_m"] < 2.0]`?

- ☐ Новый DataFrame только из строк, где глубина меньше 2 м
- ☐ Исходный DataFrame с удалёнными навсегда строками
- ☐ Колонку True/False без самих данных

- ☐ Число строк, где глубина меньше 2 м

3. На карте точек (scatter) форма водоёма выглядит сплюснутой. Наиболее вероятная причина?

- ☐ В данных перепутаны местами глубина и координата
- ☐ Выбрана неудачная цветовая шкала
- ☐ Не задан равный масштаб осей — нужен `ax.set_aspect("equal")`
- ☐ Точек слишком много для scatter

4. Какое правило нарушает команда `df.to_csv("bathymetry-training-80k.csv")` после чистки?

- ☐ CSV нельзя использовать для хранения числовых данных
- ☐ Метод `to_csv` не умеет сохранять очищенные данные
- ☐ После чистки сохранять данные вообще не нужно
- ☐ Сырые данные неприкосновенны: очищенную версию сохраняют новым файлом, не перезаписывая исходник

Проверить

Контрольные вопросы

1. Назовите три команды первичного осмотра DataFrame и что именно проверяет каждая. (*Bloom: знать*)
2. Объясните разницу между «долей мелководий по точкам съёмки» и «долей мелководий по площади водоёма» и почему pandas-фильтром получается только первая. (*Bloom: понимать/анализировать*)
3. Составьте план чистки для CSV-журнала взвешиваний вашего хозяйства: какие проверки в каком порядке и что попадёт в протокол. (*Bloom: применять*)

Что дальше?

Таблицы освоены — но половина данных специалиста живёт в другой форме: спутниковые снимки и сетки глубин — это растры. В [лекции 3 «Открываем растры и автоматизируем рутину»](#) вы прочитаете GeoTIFF как числовой массив, посчитаете по нему NDVI и напишете первый пакетный скрипт — один код на папку файлов. А [практикум](#) уже ждёт: конвейер этой лекции — его сердцевина.

Открываем растры и автоматизируем рутину: rasterio и пакетная обработка

🕒 **Трудозатраты:** ~18 мин чтения + ~30 мин практики

Модуль **П4: Python и Jupyter для специалистов**, лекция 3 из 3. Профессиональный уровень. Проект UNDP-KAZ, FOLUR.

Цели лекции и место в модуле

Таблицы из [лекции 2](#) — половина мира данных; вторая половина — растры: спутниковые снимки, сетки глубин, карты индексов. Эта лекция показывает, что растр для Python — просто числовой массив, и открывает его теми же руками: NumPy для чисел, rasterio для файлов GeoTIFF. Финал лекции — навык, ради которого специалисты и учат Python: **пакетная обработка** — один скрипт, обрабатывающий целую папку файлов, будь то серия снимков за сезон или выгрузки эхолота по нескольким водоёмам.

После изучения лекции слушатель сможет:

1. **[Bloom: понимать]** объяснять устройство растра (пиксели, каналы, разрешение, nodata) и почему GeoTIFF читается как массив NumPy.
2. **[Bloom: применять]** открывать GeoTIFF в rasterio, читать метаданные и канал, маскировать значения по физическому диапазону и считать статистики и площади по ячейкам.
3. **[Bloom: применять]** вычислять NDVI по каналам Sentinel-2 и строить изображение растра в matplotlib.
4. **[Bloom: анализировать]** проектировать пакетный конвейер: функция-обработчик + цикл по файлам + сводная таблица результатов + защита от сбоя на одном файле.

Границы лекции. Внутри: растр как массив, rasterio, NDVI по готовым каналам, пакетная обработка файлов. Снаружи: базовый Python — в [лекции 1](#); таблицы — в [лекции 2](#); поиск и заказ снимков, интерпретация NDVI по сезонам — в [П7](#) и [П3](#); проекции, привязка и настоящая картография — в [П2/A2](#); интерполяция промеров в сетку глубин — в [A4](#). Здесь мы **читаем и считаем** растры, а не создаём их.

1. Растр — это массив чисел с паспортом

1.1 NumPy за пять минут

NumPy — библиотека числовых массивов, фундамент всего анализа данных в Python (pandas построен на ней). Массив похож на список, но все операции векторные — применяются ко всем элементам сразу, без цикла:

```
import numpy as np

a = np.array([0.27, 2.87, 3.4, 16.1])
a * 2          # каждый элемент удвоен
a > 2.0        # булев массив — как маски в pandas
a[a > 2.0].mean() # средняя по элементам глубже 2 м
```

Двумерный массив — таблица чисел `arr[строка, столбец]`. Именно так устроен растр: сетка пикселей, в каждом — число (глубина, яркость канала, значение индекса). Всё, что вы умеете с масками из лекции 2, работает и здесь — синтаксис тот же.

Почему не обойтись циклами из лекции 1? Из-за масштаба: фрагмент снимка Sentinel-2 на один сельский округ — это миллионы пикселей, а цикл Python обходит их поэлементно и мучительно долго. NumPy исполняет ту же операцию во внутреннем оптимизированном коде сразу над всем массивом — векторно, — и разница во времени составляет порядки. Отсюда практическое правило растровой работы: **если вы пишете цикл по пикселям — остановитесь и поищите векторную операцию**; почти всегда она есть и записывается короче.

У массива есть два свойства-паспорта, которые вы будете проверять постоянно:

`arr.shape` — размерность (строки, столбцы), `arr.dtype` — тип чисел. Несовпадение `shape` двух массивов, которые вы собираетесь складывать или делить, — типовая ошибка (например, каналы разных разрешений); проверка `shape` перед арифметикой — секунда, а ловит она половину растровых недоразумений.

1.2 Что в «паспорте» растра

GeoTIFF — это массив плюс метаданные: размер в пикселях, число каналов, тип чисел (`float32`, `uint16` ...), геопривязка (где сетка лежит на Земле и каков размер пикселя) и специальное значение **nodata** — код «здесь данных нет». Библиотека **rasterio** читает и то и другое:

```
!pip install rasterio          # в Colab — один раз за сессию
import rasterio

src = rasterio.open("bathymetry-grid-100m.tif")
print(src.width, src.height)   # 211 x 166 пикселей
```

```
print(src.count, src.dtypes) # 1 канал, float32
print(src.nodata)          # None – маски nodata нет!
arr = src.read(1)          # канал 1 -> массив NumPy 166x211
```

Наш пример — открытая **сетка глубин 100 м** (уровень В политики данных, `bathymetry-grid-100m.tif`): производный обобщённый продукт того же водохранилища степной зоны Казахстана, промеры которого вы чистили в лекции 2. Каждый пиксель — 100×100 м, значение — глубина в метрах.

1.3 Урок честности: не всё, что в файле, — данные

Выведите статистики массива — и получите сюрприз: минимальные значения глубоко отрицательные, максимальные — заметно больше 16,1 м, хотя таких глубин в водоёме нет. Причина в паспорте: `nodata = None`. Сетка построена интерполяцией по прямоугольнику, накрывающему водоём, и в ячейках **за пределами акватории** интерполятор экстраполировал — досочинил значения, не имеющие физического смысла. Файл честен по-своему: он нигде не утверждает, что все ячейки — вода; но специалист, посчитавший «среднюю глубину» по всему массиву, получит бессмыслицу.

Лечение — маска по физическому диапазону, знакомое движение из лекции 2:

```
voda = arr[(arr >= 0) & (arr <= 16.1)] # только правдоподобные ячейки
print(f"Ячеек воды: {voda.size}")
print(f"Средняя глубина: {voda.mean():.2f} м")

ploshchad_km2 = voda.size * 100 * 100 / 1e6 # ячейка 100x100 м
obem mln_m3 = voda.sum() * 100 * 100 / 1e6 # метод призм: сумма глубин x
# площадь ячейки
print(f"Площадь ≈ {ploshchad_km2:.1f} км², объём ≈ {obem mln_m3:.1f} млн м³")
```

Три строки — и у вас оценка площади и объёма водоёма (метод призм: объём = сумма по ячейкам глубины, умноженной на площадь ячейки). Это **оценка по обобщённой сетке с грубой маской**, а не гидрографический расчёт: у настоящей обработки маска акватории строится по береговой линии, а не по диапазону значений — границы применимости обсудим в конце. Но принцип «открой паспорт → проверь nodata → маскируй → считай» — универсален для любого растра, который вам пришлют.

Нарисовать растр — одна команда matplotlib:

```
import matplotlib.pyplot as plt

masked = np.where((arr >= 0) & (arr <= 16.1), arr, np.nan) # вне воды -> NaN
fig, ax = plt.subplots(figsize=(9, 7))
im = ax.imshow(masked, cmap="viridis_r")
fig.colorbar(im, label="Глубина, м")
ax.set_title("Сетка глубин 100 м (уровень В)")
fig.savefig("setka-glubin.png", dpi=150)
```

`np.where(условие, значение_да, значение_нет)` заменяет ячейки вне диапазона на `NaN` — `matplotlib` рисует их прозрачными, и на экране остаётся только акватория.

2. NDVI из каналов Sentinel-2: растровая арифметика

2.1 Откуда берутся файлы

Снимки Sentinel-2 бесплатны и открыты. Для специалиста без ГИС-подготовки самый простой путь — **Copernicus Browser** (browser.dataspace.copernicus.eu): найти свой участок, выбрать безоблачную дату, выгрузить нужные каналы как GeoTIFF. Альтернативы — Google Earth Engine и Microsoft Planetary Computer (работа с их каталогами — навык модулей [A3](#) и [П7](#)). Для NDVI нужны два канала: **B04** (красный) и **B08** (ближний инфракрасный), оба с разрешением 10 м.

Пара слов о размере. Полный тайл Sentinel-2 — почти 11 тысяч пикселей по стороне при разрешении 10 м, и в `float32` один такой канал занимает сотни мегабайт памяти; несколько подобных массивов способны исчерпать память сессии Colab.

Практический приём: выгружайте из Copernicus Browser не весь тайл, а фрагмент по границе своего участка — интерфейс позволяет обвести область интереса, массивы получаются в сотни раз меньше, и весь конвейер лекции отрабатывает мгновенно. Если массив всё-таки велик, `rasterio` умеет читать растр «по окнам» (`windowed reading`) — тема за рамками модуля, но термин для поиска у вас теперь есть.

Две детали выбора, которые экономят часы. Первая — уровень обработки: берите продукты **L2A** (отражение у поверхности, атмосферная коррекция уже выполнена поставщиком) — для сравнения дат между собой это существенно. Вторая — облачность: даже лёгкая дымка или полупрозрачное перистое облако искажают индекс, а тень от облака на поле выглядит на NDVI-карте как «проблемная зона», которой нет. Правило специалиста: сначала посмотреть сцену глазами в браузере, потом выгружать. Пиксель 10 м означает, что на поле в сотни гектаров приходится десятки тысяч пикселей — достаточно и для среднего по полю, и для карты неоднородности; а вот узкая лесополоса или край поля шириной в один-два пикселя всегда будут «смешанными» — интерпретировать их значения нужно осторожно.

2.2 Формула в четыре строки

$$NDVI = (NIR - RED) / (NIR + RED)$$
 — нормализованный вегетационный индекс: живая листва сильно отражает ближний инфракрасный свет и поглощает красный, поэтому у активной растительности NDVI высок (ближе к 1), у открытой почвы — низок, у воды — отрицателен. Смысл и сезонное поведение индекса подробно разобраны в [П3](#) и [П7](#); здесь — как посчитать его самому:

```
red = rasterio.open("B04.tif").read(1).astype("float32")
nir = rasterio.open("B08.tif").read(1).astype("float32")

ndvi = (nir - red) / (nir + red + 1e-9)  # +1e-9 спасает от деления на 0
print(f"Средний NDVI участка: {np.nanmean(ndvi):.2f}")
```

Арифметика с массивами поэлементна: одна строка формулы обсчитывает миллионы пикселей. Мелкая, но важная деталь — `astype("float32")`: каналы Sentinel-2 хранятся целыми числами, и без преобразования деление целых даст мусор. Добавка `1e-9` в знаменатель защищает от деления на ноль в пустых пикселях. Результат `ndvi` — такой же массив: его можно маскировать, усреднять по зонам и рисовать `imshow`, как сетку глубин в §1.3.

Проверка на здравый смысл обязательна и здесь: NDVI по построению лежит в диапазоне $-1 \dots +1$. Значения за пределами — признак ошибки (перепутаны каналы, не сделан `astype`). Привычка «посчитал → проверь диапазон» — та же, что с глубинами, и она станет обязательным шагом в [ИИ-задании](#).

Подробнее о выборе дат, облачности, интерпретации NDVI-карт и переходе на съёмку БПЛА — в [П7](#); о готовых сервисах NDVI-мониторинга без кода — в [П3](#).

3. Пакетная обработка: один скрипт — вся папка

3.1 Задача, ради которой всё затевалось

Методология платформы называет этот навык прямо: «пакетная обработка батиметрических данных». Реальные сценарии: эхолот выгружает съёмку **несколькими CSV-файлами** (по дням или галсам); у хозяйства **несколько водоёмов** — по каждому свой файл промеров; мониторинг поля даёт **серию NDVI-файлов за сезон**. Обработать их по одному, меняя имя файла руками, — значит потерять главное преимущество кода. Правильная конструкция всегда одна и та же: **функция-обработчик + цикл по файлам + сводная таблица**.

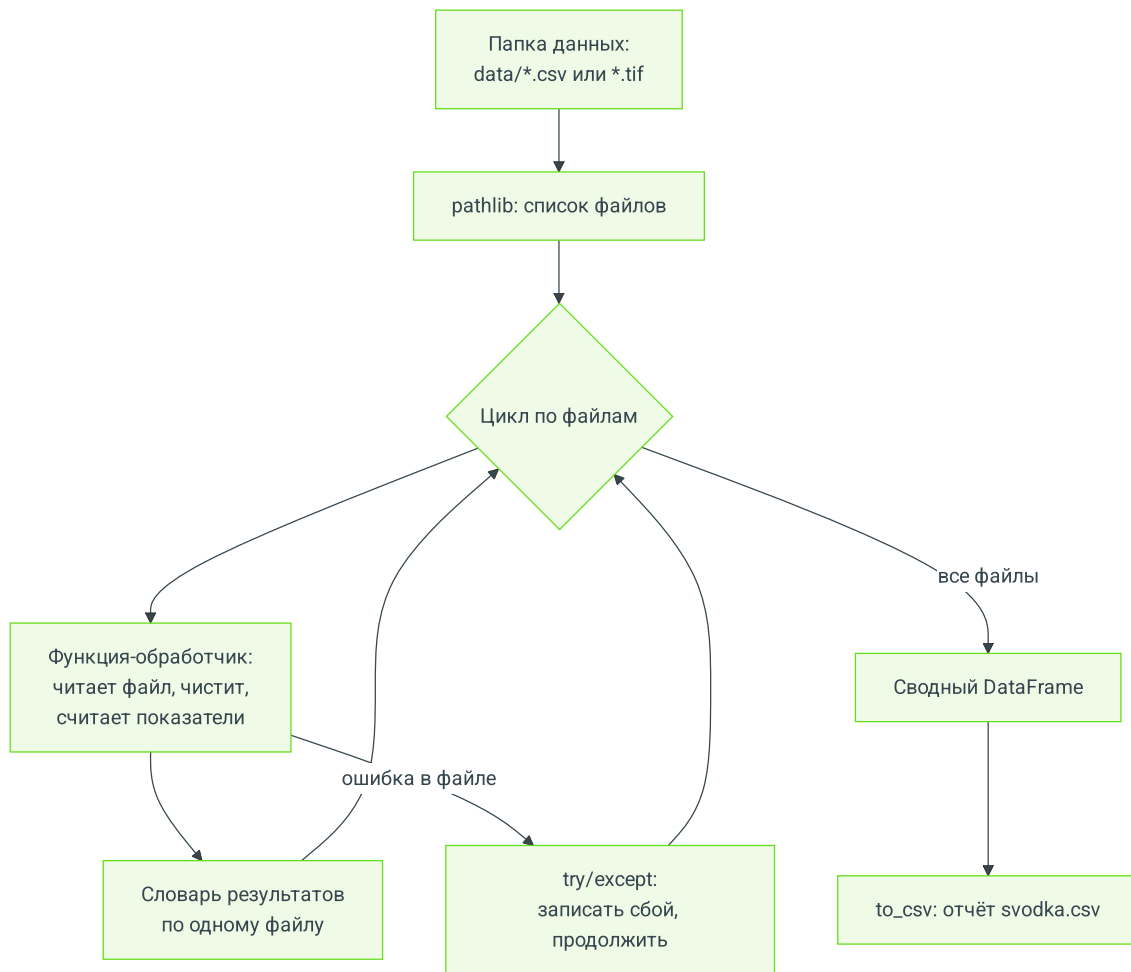


Рис. 2. Пакетный конвейер: функция-обработчик применяется к каждому файлу папки, сбойный файл не останавливает остальные, результаты собираются в сводную таблицу. Alt: блок-схема — список файлов из папки поступает в цикл, каждый файл обрабатывается функцией, ошибки перехватываются, итог сводится в DataFrame и сохраняется в CSV.

3.2 Скелет пакетного скрипта

```

from pathlib import Path
import pandas as pd

def passport_vodoema(fajl):
    """Показатели одного CSV с промерами: имя, число точек, чистка,
    статистики."""
    df = pd.read_csv(fajl)
    was = len(df)
    clean = df[(df["depth_m"] >= 0) & (df["depth_m"] <= 17.0)]
    return {
        "fajl": fajl.name,
        "tochek": was,
        "udaleno": was - len(clean),
        "sred_glubina": round(clean["depth_m"].mean(), 2),
        "maks_glubina": clean["depth_m"].max(),
    }

```

```

        "dolya_melkovodij": round((clean["depth_m"] < 2).mean(), 3),
    }

rezultaty = []
for f in sorted(Path("data").glob("*.csv")):    # все CSV в папке data/
    try:
        rezultaty.append(pasport_vodoema(f))
    except Exception as e:                      # сбойный файл не валит весь
прогон
        rezultaty.append({"fajl": f.name, "oshibka": str(e)})

svodka = pd.DataFrame(rezultaty)
svodka.to_csv("svodka.csv", index=False)
svodka

```

Разберём несущие элементы. `Path("data").glob("*.csv")` возвращает все CSV-файлы папки — добавили в папку десятый файл, скрипт обработает и его без единой правки. Функция `pasport_vodoema` — весь конвейер лекции 2, упакованный по правилу «функция = переиспользуемый инструмент» из лекции 1. Конструкция `try/except` — страховка пакетного прогона: один битый файл (пустой, с другими колонками) записывается в отчёт строкой с ошибкой, а остальные обрабатываются. Финал — сводный DataFrame: строка на файл, колонки-показатели, и это уже готовая таблица в отчёт руководителю.

Тот же скелет обрабатывает растры: замените `*.csv` на `*.tif`, а внутри функции — `pd.read_csv` на `rasterio.open(...).read(1)` и статистики §1.3. Скелет не меняется — меняется начинка функции. Это и есть признак выученного навыка: вы больше не пишете скрипты с нуля, вы подставляете обработчик в готовую конструкцию.

3.3 Порядок в папке — половина пакетной обработки

Пакетный скрипт хорош настолько, насколько предсказуемы имена и структура файлов. Три соглашения, которые стоит завести в хозяйстве до всякого кода. **Имена файлов несут данные:** `pole1_2026-06-01_B04.tif` разбирается кодом на поле, дату и канал; файл `новый снимок (копия 2).tif` — не разбирается ничем. Дату пишете в формате `ГГГГ-ММ-ДД` — такие имена корректно сортируются по алфавиту. **Структура папок фиксирована:** сырые файлы — в `data/`, результаты — в `output/`, скрипт не пишет ничего в `data/`. **Одна папка — один тип файлов:** если в `data/` вперемешку лежат CSV промеров и экселевские сводки, шаблон `*.csv` защитит скрипт, но лучше не создавать ситуацию вовсе. Эти правила — прямое продолжение системы учёта из модуля П1: пакетная обработка — первый потребитель наведённого там порядка.

3.4 Протокол и воспроизводимость в пакетном режиме

Пакетный скрипт обязан отчитываться: сколько файлов найдено, сколько обработано, сколько сбоев, сколько строк удалено чисткой в каждом. Всё это уже

есть в сводной таблице — приучите себя смотреть её **до** содержательных колонок: колонка `udaleno` с аномально большим значением укажет на файл с проблемой данных быстрее любой гистограммы. И, как всегда: сырые файлы в `data/` неприкосновенны, все результаты — новыми файлами, финальная проверка ноутбука — Restart & Run all (лекция 1, §4.3).

Когда пакетный конвейер перестаёт помещаться в ноутбук — файлов сотни, обработка занимает часы, снимки нужно не только считать, но и искать по каталогу автоматически, — это признак, что вы переросли уровень П4. Дальше два маршрута: Google Earth Engine, где каталог и вычисления живут в облаке (A3, П7), либо полноценные скрипты вне ноутбука. Хорошая новость: конструкция «функция + цикл + сводка» остаётся той же на любом уровне — меняются только инструменты вокруг неё.

Региональный кейс

Ситуация: Техническому специалисту зернового хозяйства в Северо-Казахстанской области поставили задачу: по трём проблемным полям отслеживать состояние посевов в течение сезона. Специалист раз в две недели выгружает из Copernicus Browser безоблачные фрагменты Sentinel-2 (каналы B04 и B08) по каждому полю — к середине сезона накапливается папка из десятков GeoTIFF, и ручной расчёт в калькуляторе индексов уже не масштабируется. Нужен пакетный скрипт: по каждому файлу дата и поле — средний NDVI, на выходе — таблица динамики.

Данные: Sentinel-2 L2A, каналы B04/B08 по выбранным полям (Copernicus Data Space / Browser, бесплатно); имена файлов вида `pole1_2026-06-01_B04.tif`.

Задача:

1. Написать функцию `sredniy_ndvi(fajl_b04, fajl_b08)`: чтение каналов, `astype("float32")`, формула NDVI (§2.2), проверка диапазона $-1...+1$, возврат среднего.
2. Собрать пакетный цикл по парам файлов папки (скелет §3.2) со страховкой `try/except`; дату и имя поля извлечь из имени файла.
3. Свести результаты в DataFrame «поле × дата → средний NDVI», сохранить CSV и построить график динамики по каждому полю.

Разбор: Ожидаемый ход: функция возвращает словарь `{поле, дата, ndvi_sred}`, цикл наполняет список, DataFrame сводит и рисует три линии динамики. Конкретные значения NDVI зависят от полей, культур и дат — их вычисляет слушатель; проверки правдоподобия: все значения в $-1...+1$, у вегетирующих посевов летом индекс существенно выше, чем у открытой почвы весной, а резкий провал одной даты на

всех трёх полях сразу — почти наверняка облачность или дефект сцены, а не агрономия. Интерпретация провалов по одному полю (проблемная зона? уборка?) — предмет П7: наш скрипт — измерительный прибор, выводы делает агроном.

Закрепление и применение

Попробуйте в Colab

Ноутбук лекции (<URL будет назначен при создании репозитория ноутбуков>) содержит код §1–§3. Минимальное задание на 20 минут: откройте сетку глубин уровня В, воспроизведите «урок честности» §1.3 — статистики до и после маскирования — и получите свою оценку площади и объёма; затем прогоните пакетный скелет §3.2 на папке с двумя-тремя копиями учебного CSV.

Границы применимости

Маска «0–16,1 м» — учебное приближение: настоящая маска акватории строится по береговой линии, а расчёт объёма — по методике гидрографии (A4); наши числа — оценка по обобщённой сетке 100 м. NDVI из §2.2 корректен для сцен L2A без облаков; сравнение дат требует одинаковой обработки. Сетка уровня В — обобщённый открытый продукт: реальная геопривязка полного датасета промеров относится к уровню С и доступна только как дополнительный материал после юридического подтверждения (см. Датасеты).

Что это даёт вашему хозяйству

Пакетный скелет §3.2 — самый переносимый фрагмент модуля: папка выгрузок весовой, папка журналов по бригадам, папка снимков по полям — один и тот же цикл со своей функцией-обработчиком. Настроив его один раз, вы получаете еженедельную сводку по всем объектам хозяйства ценой запуска одной ячейки.

Кейс для размышления

Подрядчик прислал «карту глубин вашего пруда» одним GeoTIFF без пояснений. Опираясь на §1.2–1.3, составьте список из четырёх проверок паспорта и значений файла, которые вы выполните до того, как поверить хоть одному числу из этой карты. Что вы сделаете, если nodata не задан?

Словарь урока

Термин	Значение
Растр	сетка пикселей со значениями; для Python — двумерный массив NumPy
Канал (band)	один слой растра; у Sentinel-2 каналы соответствуют диапазонам спектра (B04 — красный, B08 — ближний ИК)
GeoTIFF	формат «массив + паспорт»: размер, число каналов, тип чисел, геопривязка, nodata
nodata	служебное значение «данных нет»; если не задано — в файле могут жить нефизичные ячейки
rasterio	библиотека чтения/записи GeoTIFF: открывает паспорт и каналы как массивы NumPy
Векторная операция	действие сразу над всем массивом без цикла по элементам; основа скорости NumPy
Маска по диапазону	фильтр <code>(arr >= a) & (arr <= b)</code> , оставляющий только физически правдоподобные значения
NDVI	$(\text{NIR} - \text{RED}) / (\text{NIR} + \text{RED})$; лежит в $-1 \dots +1$, у активной растительности высок
L2A	уровень обработки Sentinel-2 с атмосферной коррекцией — отражение у поверхности
glob	шаблонный отбор файлов папки: <code>Path("data").glob("*.csv")</code>
try/except	перехват ошибки: сбойный файл фиксируется в отчёте, пакетный прогон продолжается

Проверь себя

Самопроверка · вариант слушателя **конспект** · вопросов: 4 · порог зачёта: 75%

1. Чем пакетный скрипт со структурой «функция + цикл по Path.glob + сводный DataFrame» лучше десятикратного ручного запуска одного скрипта с правкой имени файла?

- ☐ Он позволяет не проверять качество данных в файлах
- ☐ Он не требует библиотеки pandas
- ☐ Новые файлы обрабатываются без правки кода, результаты сводятся в одну таблицу, а прогон воспроизводим и протоколируем
- ☐ Он работает быстрее на каждом отдельном файле

2. В GeoTIFF сетки глубин nodata = None, а статистика по всему массиву даёт отрицательные «глубины». Что это означает и что делать?

- ☐ Файл повреждён при скачивании — нужно скачать заново
- ☐ Ячейки вне акватории содержат экстраполяцию без физического смысла — перед расчётами нужна маска по правдоподобию диапазону
- ☐ Отрицательные значения — это высоты берега, их надо прибавить к глубинам
- ☐ rasterio неправильно читает float32 — нужно читать через pandas

3. Какова роль try/except в пакетном цикле по файлам?

- ☐ Один сбойный файл фиксируется в отчёте строкой с ошибкой, а обработка остальных продолжается
- ☐ Отключение предупреждений pandas в выводе
- ☐ Ускорение обработки за счёт пропуска больших файлов
- ☐ Автоматическое исправление битых файлов

4. Средний NDVI по сцене получился 4,7. Первое действие специалиста?

- ☐ Сменить стар на более контрастную
- ☐ Уменьшить площадь участка и пересчитать
- ☐ Искать ошибку в коде: NDVI по построению лежит в $-1 \dots +1$, значение вне диапазона — признак перепутанных каналов или пропущенного astype
- ☐ Принять результат: посевы в отличном состоянии

Проверить

Контрольные вопросы

1. Из чего состоит «паспорт» GeoTIFF и какие три поля вы проверяете первыми? (*Bloom: знать*)
2. Объясните, почему оценка объёма по сетке 100 м с маской по диапазону — приближение, и что делает её гидрографически корректной. (*Bloom: понимать/анализировать*)
3. Спроектируйте пакетный конвейер для папки из 12 CSV-выгрузок эхолота по трём прудам хозяйства: функция, цикл, сводка, страховка. (*Bloom: применять*)

Что дальше?

Все три инструмента в руках: базовый Python, pandas с matplotlib, rasterio с пакетным циклом. Пора собрать из них прикладной продукт модуля — в [практикуме «Первый рабочий скрипт»](#) вы построите полный паспорт водоёма из 80 000 промеров и адаптируете скрипт под собственный датасет. Затем — [ИИ-задание](#): тот же класс задач, но код пишет ИИ-напарник, а верифицируете вы. Продолжение по темам: обработка собственных промеров — [П11](#), мониторинг полей — [П7](#).

Практикум: первый рабочий скрипт — паспорт водоёма из 80 000 промеров

Модуль **П4: Python и Jupyter для специалистов**. Время выполнения: **~75 мин** (45 мин базовая часть + 30 мин адаптация под свои данные). Инструменты: Google Colab (браузер, бесплатный аккаунт Google) — только открытый стек. Данные: `datasets/bathymetry-training-80k.csv` — реальные эхолотные промеры водохранилища степной зоны Казахстана, уровень А политики данных платформы ([Датасеты](#)).

Цель практикума

[Bloom: применять] Самостоятельно собрать воспроизводимый Colab-ноутбук «паспорт водоёма»: загрузка сырого CSV → протоколируемая чистка → статистики и зоны глубин → гистограмма и карта точек → сохранённые артефакты (очищенный CSV + два PNG) — и адаптировать этот скрипт под собственный датасет.

Практикум собирает воедино [лекцию 1](#) (Colab, синтаксис), [лекцию 2](#) (pandas, matplotlib) и элементы [лекции 3](#) (функция-обработчик). Это и есть прикладной продукт модуля: **первый рабочий Python-скрипт под собственный датасет**.

Что нужно ПОДГОТОВИТЬ

- [] Браузер и аккаунт Google (для Colab); установка ПО не требуется.
- [] Учебный датасет `bathymetry-training-80k.csv` — скачать со страницы [Датасеты платформы](#) (файл `datasets/bathymetry-training-80k.csv` репозитория платформы).
- [] (Для шага 7) Любой ваш CSV: журнал урожайности, выгрузка весовой, показания датчиков. Если своего файла нет — шаг 7 выполняется на копии учебного.
- [] Ноутбук-заготовка: `<URL будет назначен при создании репозитория ноутбуков>` (можно работать и с чистого листа — весь код приведён ниже).

Задание

Шаг 1: Создаём ноутбук и загружаем данные (≈7 мин)

Откройте colab.research.google.com → «Файл → Создать блокнот». Переименуйте в `passport-vodoema`. Первой сделайте **текстовую** ячейку: название, ваша фамилия, дата, одно предложение о задаче — ноутбук читают как документ. Затем перетащите скачанный CSV в панель файлов слева (значок папки).

```
import pandas as pd
import matplotlib.pyplot as plt

RAW = "bathymetry-training-80k.csv"      # имя исходного файла — единственная
настройка
df = pd.read_csv(RAW)
df.head()
```

Ожидаемый результат: таблица из 5 строк с колонками `point_id`, `x_m`, `y_m`, `depth_m`. Если `FileNotFoundError` — файл не загружен в сессию или имя не совпадает (лекция 1, §4.2).

Шаг 2: Осмотр — три команды и вывод словами (≈8 мин)

```
print(df.shape)
df.info()
df.describe()
```

Под кодом добавьте **текстовую ячейку** и запишите своими словами: сколько записей; есть ли пропуски; диапазон, среднее и медиану глубины; что говорит соотношение среднего и медианы (лекция 2, §1.2).

Ожидаемый результат: 80 000 строк, 4 колонки, без пропусков; глубины от 0 до 16,1 м; среднее ≈3,4 м заметно больше медианы ≈3,0 м → распределение скошено, мелководий больше.

Шаг 3: Чистка с протоколом (≈10 мин)

```
was = len(df)

dubli = df.duplicated().sum()
df = df.drop_duplicates()

clean = df[(df["depth_m"] >= 0) & (df["depth_m"] <= 17.0)].copy()

nuli = (clean["depth_m"] == 0).sum()

print("=== Протокол чистки ===")
print(f"Исходных строк:      {was}")
print(f"Дубликатов удалено:    {dubli}")
print(f"Вне диапазона удалено: {was - dubli - len(clean)}")
print(f"Нулевых отсчётов:      {nuli} ({nuli/len(clean):.1%}) — сохранены и
учтены")
print(f"Строк в чистой версии: {len(clean)}")
```

```
clean.to_csv("bathymetry-clean.csv", index=False)
```

Обратите внимание: нули мы **не удаляем**, а документируем (лекция 2, §2.1), и результат пишем **новым файлом** — исходник неприкосновенен.

Ожидаемый результат: протокол напечатан; на учебном датасете дубликатов и значений вне диапазона нет (это тоже результат проверки!); файл `bathymetry-clean.csv` появился в панели файлов.

Шаг 4: Показатели паспорта (≈10 мин)

```
clean["zona"] = pd.cut(clean["depth_m"],
                        bins=[-0.01, 2, 5, 10, 17],
                        labels=["0-2 м", "2-5 м", "5-10 м", ">10 м"])
tablica_zon = clean.groupby("zona", observed=True)["depth_m"].agg(["count",
                           "mean"]).round(2)
print(tablica_zon)

print(f"Максимальная глубина: {clean['depth_m'].max()} м")
print(f"Доля точек мельче 2 м: {(clean['depth_m'] < 2).mean():.1%}")
print(f"Доля точек глубже 10 м: {(clean['depth_m'] > 10).mean():.1%}")
```

Ожидаемый результат: таблица «зона → число точек, средняя глубина» и три показателя. Проверка здравым смыслом: сумма `count` по зонам равна числу строк чистой версии; доли лежат между 0 и 100 %.

Шаг 5: Графики (≈10 мин)

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(clean["depth_m"], bins=40, edgecolor="white")
ax.set_xlabel("Глубина, м"); ax.set_ylabel("Число промеров")
ax.set_title("Распределение глубин (80 000 промеров)")
fig.savefig("gistogramma-glubin.png", dpi=150)

fig, ax = plt.subplots(figsize=(9, 7))
sc = ax.scatter(clean["x_m"], clean["y_m"], c=clean["depth_m"], s=1,
                cmap="viridis_r")
fig.colorbar(sc, label="Глубина, м")
ax.set_xlabel("x, м (локальная СК)"); ax.set_ylabel("y, м (локальная СК)")
ax.set_aspect("equal")
ax.set_title("Карта точек съёмки, цвет – глубина")
fig.savefig("karta-tochek.png", dpi=150)
```

Ожидаемый результат: гистограмма со скосом вправо и пиком у нуля; карта точек с видимыми галсами съёмки и без искажения формы (`set_aspect("equal")`). Оба PNG — в панели файлов.

Шаг 6: Упаковка в функцию + Restart & Run all (≈8 мин)

Соберите шаги 3–4 в функцию-обработчик по образцу лекции 3 (§3.2):

```
def passport(fajl, d_min=0, d_max=17.0, porog_melko=2.0):  
    """Паспорт одного CSV с промерами: чистка по диапазону + ключевые  
    показатели."""  
    df = pd.read_csv(fajl).drop_duplicates()  
    clean = df[(df["depth_m"] >= d_min) & (df["depth_m"] <= d_max)]  
    return {  
        "fajl": fajl,  
        "tochek": len(df),  
        "udaleno": len(df) - len(clean),  
        "sred": round(clean["depth_m"].mean(), 2),  
        "maks": clean["depth_m"].max(),  
        "dolya_melko": round((clean["depth_m"] < porog_melko).mean(), 3),  
    }  
  
print(passport(RAW))
```

Затем — обязательная проверка воспроизводимости: **«Среда выполнения → Перезапустить и выполнить всё»** (Restart & Run all). Ноутбук должен пройти сверху донизу без ошибок.

Ожидаемый результат: словарь показателей, совпадающих с шагом 4; полный прогон без ошибок.

Шаг 7: Скрипт под ВАШ датасет (≈20 мин)

Замените `RAW` на имя собственного CSV и адаптируйте: имя числовой колонки (`depth_m` → например, `urozhaynost_c_ga`), физический диапазон в фильтре, границы зон `pd.cut`, подписи графиков. Карту точек стройте, только если в данных есть координаты; иначе достаточно гистограммы. Зафиксируйте в текстовой ячейке: что за данные, какой диапазон считаете физически возможным и почему.

Ожидаемый результат: тот же конвейер печатает протокол чистки и паспорт по вашим данным.



Применить к своему хозяйству

Проект на ВАШЕЙ земле. Шаг 7 — не довесок, а главный результат модуля: скрипт, который вы сможете запускать каждый сезон. Подходит любой табличный учёт: журнал урожайности по полям, взвешивания, датчики влажности, топливные выгрузки, собственные эхолотные промеры пруда (формат совпадает с учебным). Если данные хозяйства конфиденциальны — обезличьте их перед загрузкой в Colab (уберите названия и привязку, как это сделано в учебном датасете) или выполните шаг на локальном JupyterLab (лекция 1, §2.3). Поделиться результатом: экспортируйте ноутбук («Файл → Скачать → .ipynb») — это shareable-артефакт для коллег и сертификации.

Продукт задания

По итогам практикума у вас должны быть:

- [] Ноутбук `pasport-vodoema.ipynb` с текстовыми пояснениями, проходящий Restart & Run all;
- [] `bathymetry-clean.csv` — очищенная версия (исходник не перезаписан);
- [] `gistogramma-glubin.png` и `karta-tochek.png` с подписанными осями;
- [] напечатанный протокол чистки и паспорт-словарь функции `pasport()` ;
- [] адаптация под собственный CSV (шаг 7) с пояснением выбранного диапазона.

Самопроверка: число строк чистой версии + удалённые = исходные 80 000; все глубины чистой версии в пределах 0–16,1 м; сумма точек по зонам равна числу строк; доля мелководий — между 0 и 1; карта точек не искажена (масштаб осей равный); повторный полный прогон даёт те же числа.

Дополнительные задания (по желанию)

1. **Растровый паспорт (лекция 3):** откройте сетку глубин уровня В `bathymetry-grid-100m.tif` через rasterio, примените маску правдоподобного диапазона и сравните среднюю глубину по сетке со средней по точкам. Подумайте, почему значения различаются (неравномерность галсов vs равномерная сетка).
2. **Пакетный режим:** сделайте в папке `data/` три копии учебного CSV, прогоните цикл из лекции 3 (§3.2) с функцией `pasport()` и получите `svodka.csv` .
3. **Сезонный вариант:** прогоните конвейер по CSV NASA POWER из кейса лекции 1 — паспорт влагообеспеченности сезона вместо паспорта водоёма.

Связи

- [Лекция 1](#) — Colab, синтаксис, ошибки; [Лекция 2](#) — pandas и графики (ядро практикума); [Лекция 3](#) — функция-обработчик и пакетный цикл.
- [ИИ-задание модуля](#) — тот же класс задач, но код пишет ИИ-напарник, а вы верифицируете.
- Продолжение: [П11](#) — профессиональная обработка собственных промеров; [A4](#) — интерполяция и карта глубин из этих же данных.
- Политика данных: [Датасеты платформы](#).

ИИ-ассистент в работе: первый скрипт с ИИ-напарником

Модуль **П4: Python и Jupyter для специалистов** · Уровень ИИ-слоя: **Low-code** · ~45 мин
Инструменты: Google Colab + LLM-ассистент (встроенный помощник Colab или любой чат-ассистент — Claude и аналоги; каждый инструмент заменяем открытым аналогом).

⚠️ Главное правило: ИИ ускоряет — человек верифицирует

Результат ИИ без независимой проверки — не результат. Всё задание ниже выполнимо и полностью без ИИ (код дан в **практикуме**) — ИИ-слой ускоряет работу, но не является условием прохождения модуля.

Цель задания

Освоить рабочий цикл специалиста уровня low-code: **сформулировать задачу по-русски → получить код от LLM-ассистента → исполнить в Colab → верифицировать результат по контрольным точкам**. После задания вы сможете поручать ИИ рутинное написание кода под ваши таблицы, оставляя себе постановку задачи и контроль качества — ровно то распределение ролей, которое принято в реальных командах анализа данных.

Почему постановка задачи важнее кода

LLM-ассистент пишет типовой pandas-код быстро и в основном корректно, но он не знает ваших данных: имён колонок, физического диапазона значений, того, что нули в промерах бывают законными. Всё это обязан сообщить постановщик — вы. Рабочий промпт-паттерн из четырёх частей:

1. **Роль и контекст** — «Я работаю в Google Colab, у меня CSV с эхолотными промерами»;
2. **Данные** — точные имена колонок, размер, особенности: «колонок `point_id`, `x_m`, `y_m`, `depth_m`, 80 000 строк, глубины 0–16,1 м, нулевые значения — частично законные точки берега, их удалять нельзя»;

3. **Шаги** — что именно сделать, по пунктам (чистка по диапазону → показатели → гистограмма → сохранение нового CSV);
4. **Формат ответа** — «одна ячейка кода с комментариями по-русски; исходный файл не перезаписывать».

Чем конкретнее пункты 2–3, тем меньше правок. Расплывчатый промпт «почисти мне данные» даёт код, который молча удалит ваши нули.

Пошаговое задание

Шаг 1. Подготовка (5 мин). Откройте новый Colab-ноутбук, загрузите `bathymetry-training-80k.csv` (см. [Датасеты](#)). В текстовой ячейке зафиксируйте цель: «Паспорт водоёма: чистка, показатели, гистограмма — код пишет ИИ-ассистент, верификация моя».

Шаг 2. Постановка задачи (10 мин). Составьте промпт по паттерну выше и запросите у ассистента код, который: (а) читает CSV; (б) печатает протокол чистки — число строк до/после фильтра физического диапазона 0–17 м и число дубликатов; (в) считает среднюю, медианную и максимальную глубину и долю точек мельче 2 м; (г) строит гистограмму глубин с подписанными осями; (д) сохраняет очищенную версию в **новый** файл. Сохраните текст промпта в текстовой ячейке ноутбука — промпт является частью протокола работы.

Шаг 3. Ревью перед запуском (5 мин). Прочитайте предложенный код **до** исполнения и проверьте три пункта: читается ли правильный файл; не перезаписывается ли исходник (`to_csv` только в новое имя!); не удаляются ли нулевые глубины. Нашли проблему — не правьте вручную, а сообщите ассистенту: «нули удалять нельзя, исправь» — умение добиваться правки итерациями и есть навык.

Шаг 4. Исполнение (5 мин). Запустите код. Ошибка? Скопируйте traceback целиком ассистенту и попросите исправить ([лекция 1](#), §4.2). Две-три итерации — норма.

Шаг 5. Обязательная верификация (15 мин). Проверьте результат ИИ независимо — по контрольным точкам, которые вы знаете о данных заранее:

Контрольная точка	Ожидание	Как проверить
Число исходных строк	ровно 80 000	<code>len(pd.read_csv(RAW))</code> — своя ячейка, не код ИИ
Диапазон глубин чистой версии	0...16,1 м	<code>clean["depth_m"].min(), .max()</code>

Контрольная точка	Ожидание	Как проверить
Баланс строк	удалено + осталось = исходные	арифметика по протоколу
Нулевые отсчёты	сохранены, доля $\approx 4-5\%$	<code>(clean["depth_m"] == 0).mean()</code>
Средняя и медиана	$\approx 3,4$ м и $\approx 3,0$ м — сходится с <code>df.describe()</code>	сверка с шагом 2 практикума
Воспроизводимость	полный прогон без ошибок	Restart & Run all

Любое расхождение — возврат к шагу 3: разбираетесь, чей это дефект (постановки или кода), и добиваетесь исправления. В текстовой ячейке зафиксируйте итог верификации: какие точки сошлись, сколько итераций потребовалось.

Шаг 6 (по желанию). Свои данные. Повторите цикл для собственного CSV из шага 7 практикума — теперь контрольные точки формулируете вы сами, до обращения к ИИ. Это главный перенос навыка: верификация начинается со знания собственных данных.

Критерии зачёта

- [] В ноутбуке сохранён текст промпта (постановка задачи по 4-частному паттерну);
- [] Код получен от ассистента и исполнен; итерации правок видны или описаны;
- [] Ревью до запуска выполнено: исходный файл не перезаписан, нули не удалены;
- [] Таблица верификации заполнена: все шесть контрольных точек проверены **отдельными ячейками, написанными вами**, расхождения объяснены;
- [] Ноутбук проходит Restart & Run all;
- [] Вывод в финальной текстовой ячейке: что ИИ сделал хорошо, где ошибся, что вы изменили бы в промпте.

Незачёт при любом из двух: верификация выполнена кодом самого ассистента «проверь сам себя» (это не независимая проверка) или результат принят на веру без таблицы контрольных точек.

СВЯЗИ

Рамка ИИ-слоя платформы — **ИИ-инструменты в обучении**. Расширение навыка: делегирование фильтрации промеров кодинг-агенту с верификацией по опорным глубинам — **ИИ-задание A4** (pro-code); no-code работа с готовыми ИИ-сервисами — **ПЗ**.

Итоговая аттестация модуля П4

Модуль **П4: Python и Jupyter для специалистов** · ~45 мин Формат: индивидуальный вариант из банка MCQ (тест-движок платформы) + сценарный кейс с экспертной проверкой. Порог сертификации — **75 %**. Обязательное условие допуска: сданы артефакты **практикума** (ноутбук, очищенный CSV, PNG) и **ИИ-задания** (таблица верификации).

Банк вопросов · блок 1: Colab, синтаксис, культура работы

Самопроверка · вариант слушателя **конспект** · вопросов: 4 · порог зачёта: 75%

1. Файл данных исчез из сессии Colab после перерыва в работе. Почему?

- ☐ CSV-файлы хранятся в Colab не более 10 минут
- ☐ Файл переместился в корзину аккаунта
- ☐ Сессии Colab временные: виртуальная машина обнуляется, файлы нужно загружать заново или подключать Google Диск
- ☐ Файл удалён антивирусом Google

2. Код упал с `NameError: name 'clean' is not defined`. Наиболее вероятная причина в ноутбуке?

- ☐ Файл данных повреждён
- ☐ Ячейка, где создаётся `clean`, не была исполнена в текущей сессии (или содержит опечатку в имени)
- ☐ Библиотека `pandas` не установлена
- ☐ В Colab запрещено имя `clean`

3. Какое из правил культуры работы с данными нарушает скрипт, который «чистит» файл и сохраняет результат под исходным именем?

- ☐ Функции должны иметь докстринги
- ☐ Сырые данные неприкосновенны: результат чистки всегда сохраняется новым файлом
- ☐ Комментарии должны описывать намерение

- ☐ Ноутбук должен содержать текстовые ячейки

4. По правилу «один раз — как умеете, каждую неделю — скрипт» автоматизацию в первую очередь заслуживает...

- ☐ разовое переименование трёх файлов
- ☐ ежемесячная сводка по журналам всех бригад из однотипных CSV
- ☐ единичный пересчёт одной накладной
- ☐ однократное построение схемы проезда к полю

Проверить

Банк вопросов · блок 2: pandas, matplotlib, растры, пакетная обработка

Самопроверка · вариант слушателя · конспект · вопросов: 6 · порог зачёта: 75%

1. Что делает выражение `df[(df["depth_m"] >= 2.0) & (df["depth_m"] < 5.0)]`?

- ☐ Удаляет колонку `depth_m`
- ☐ Сортирует таблицу по глубине
- ☐ Заменяет глубины вне диапазона на NaN
- ☐ Отбирает строки с глубиной от 2 включительно до 5 м двумя булевыми масками через «и»

2. `df.info()` показал, что колонка `depth_m` имеет тип `object`. О чём это говорит?

- ☐ Колонка содержит только целые числа
- ☐ Колонка зашифрована
- ☐ В колонке есть нечисловые значения (текст, мусор) — прямые расчёты дадут ошибку, нужна проверка данных
- ☐ Так pandas помечает колонку с координатами

3. В пакетном цикле по 12 файлам один файл битый. Как должен вести себя правильно спроектированный скрипт?

- ☐ Молча пропустить файл без следа в отчёте
- ☐ Перехватить исключение try/except, записать сбой строкой в сводную таблицу и обработать остальные 11 файлов
- ☐ Удалить битый файл из папки
- ☐ Остановиться на битом файле, чтобы специалист сразу его починил

4. Почему долю точек мельче 2 м из промеров нельзя напрямую выдавать за долю площади мелководий водоёма?

- ☐ Потому что глубина 2 м не бывает у степных водоёмов
- ☐ Потому что pandas не умеет считать доли
- ☐ Промеры распределены неравномерно (вдоль галсов), и статистика по точкам не равна статистике по площади — нужен переход к сетке
- ☐ Потому что доля по точкам всегда завышена ровно вдвое

5. GeoTIFF открыт в rasterio, nodata = None, в массиве есть значения -16 м и +25 м при известном диапазоне глубин 0–16,1 м. Правильное действие перед расчётом среднего?

- ☐ Прибавить 16 ко всем значениям
- ☐ Взять модуль всех значений
- ☐ Сменить тип массива на uint8
- ☐ Замаскировать массив по физическому диапазону — вне его лежит экстраполяция интерполятора за пределами акватории

6. Чем функция-обработчик (например, `rasport(fajl)`) отличается от копирования того же кода под каждый файл?

- ☐ Логика написана один раз и применяется к любому файлу параметром; правки вносятся в одном месте, конвейер масштабируется циклом
- ☐ Отличий нет — это вопрос стиля
- ☐ Функция не требует импорта pandas
- ☐ Функция выполняется на серверах Google быстрее

Проверить

Сценарный кейс (образец экзаменационной части)

Кейс проверяется экспертом; ответ — связный текст на 15–25 строк, допускаются фрагменты кода. Индивидуальные варианты генерируются тест-движком с заменой предметной области (промеры / урожайность / датчики / NDVI-серия).

Ситуация. Вы — технический специалист сельхозпредприятия в Северо-Казахстанской области. Подрядчик передал результаты эхолотной съёмки пруда-накопителя: папку из 9 CSV-файлов (по одному на день съёмки) с колонками `point_id`, `x_m`, `y_m`, `depth_m` и без какой-либо документации. Руководителю к завтрашнему совещанию нужны: суммарное число промеров, диапазон и средняя глубина, доля мелководий и одна наглядная картинка. Коллега предлагает: «Скорми всё ИИ-ассистенту, он сам разберётся».

Вопросы.

1. Спроектируйте конвейер решения: какие шаги, в каком порядке, что печатается в протокол (опирайтесь на связку «функция-обработчик + цикл + сводка»). Какие проверки данных вы заложите и какие решения (например, про нулевые глубины) зафиксируете письменно?
2. Определите роль ИИ-ассистента в этой задаче по правилам low-code-уровня: что вы поручите ему, что оставите себе и по каким контрольным точкам верифицируете его код до и после запуска?
3. Сформулируйте две оговорки, которые вы обязаны включить в ответ руководителю, чтобы результат был честным (подсказка: точки vs площадь; что базовая чистка не ловит).

Что оценивается (для самоподготовки): порядок конвейера и протокол чистки (шаги осмотра → чистка с фиксацией правил → агрегация → график → сводка по 9 файлам, сырые файлы не перезаписываются); корректное распределение ролей с ИИ (постановка по 4-частному паттерну, ревью до запуска, независимая верификация контрольными точками — число строк, диапазон, баланс удалённого, воспроизводимость); зрелость оговорок (доля мелководий по точкам — не доля площади; правдоподобные локальные выбросы базовой чисткой не ловятся, для официальной карты глубин нужна гидрографическая обработка уровня А4/П11). Изобретать конкретные числа в ответе не требуется и не оценивается — оценивается методика.

Порядок аттестации

1. Тест-движок собирает индивидуальный вариант: 8 вопросов из банка модуля (блоки 1–2) — автопроверка, порог **75 %**.
2. Сценарный кейс — экспертная проверка по критериям выше.

3. Артефакты практикума и ИИ-задания — обязательное приложение; ноутбук проверяется запуском Restart & Run all.

Успешное прохождение всех трёх частей даёт сертификат платформы (QR-верификация). Пересдача MCQ — новым индивидуальным вариантом.